EurekaZero

Engineered Intelligence for Algorithm Invention by Self-Play

JopeDime Team

Draft — October 2026

Abstract

Can experience from algorithm discovery improve the first algorithm a system proposes for a new problem? We cast invention as a game: the encoded task is the board, a complete executable program is the move, a verifier that scores the program on folds withheld from its fit is the referee, and the incumbent algorithm is the opponent. EurekaZero is a design for learning this game by self-play, after AlphaGo Zero, on tabular prediction tasks whose moves are programs in a typed language of learning primitives. During training, verified search supplies targets for a grammar-constrained program writer and for a separate problem–program ranking map. Two ratchets govern what is kept: a board's incumbent changes only on strict improvement, and a retrained writer must pass a held-out admission test. At deployment the frozen writer commits one complete program per problem (search = 1), and encoding, generation and fitting are all charged to that single proposal. We contribute the game and its rules, a cost model that links proposal quality to the cost of a verified improvement, a referee protocol that keeps sealed test families outside the learner's inputs, and an evaluation protocol that compares the single proposal both with algorithms designed by frontier models and with agents allowed repeated attempts, and tests the map's ranking against a program-only control. The development records available so far (admission and program-promotion logs from later adapter training, and earlier search and ranking diagnostics) do not establish a single-proposal advantage or transfer to unseen problem families; Section 7 specifies those tests. **[TBD: State the headline first-proposal result with its uncertainty once the independent evaluation is complete.]**

1 Can Machines Invent?

We specify how to test whether feedback from execution improves the first algorithm a system proposes for a new problem. AlphaGo Zero supplies a precedent within a fixed game: it went beyond the best human play without seeing a human game, learning from self-play under the rules of Go [102].

The skeptics' challenge. In March 2025, Yann LeCun argued that language models retrieve existing solutions rather than find new ones: "in their purest form, they are incapable of inventing new things" [58]. Thomas Wolf warned that current trends will give us "a country of yes-men on servers" [118]. Dwarkesh Patel had asked Dario Amodei in August 2023 what to make of models with "basically the entire corpus of human knowledge memorized" that had not made "a single new connection that has led to a discovery" [80]. In January 2025, Demis Hassabis posed the hardest version as a benchmark for general intelligence: today's systems "can play a game of Go at a world champion level. But could a system invent Go?" [41]. These challenges concern both proposing solutions and choosing new questions; our test concerns only algorithm proposals under human-defined tasks and metrics (Section 8). Predicting historical text does not directly reward improvement over an incumbent. That motivates a different training signal; it does not establish a ceiling on what a pretrained model can discover (Section 2).

The human challenge. In their 2020 study, Bloom et al. [8] reported rising research effort and declining productivity in the industries, products and firms they examined. Their Moore's law example required more than 18 times as many researchers as in the early 1970s to sustain chip-density growth. Jones [46] found later first inventions and greater specialization in the inventor data he studied. Park et al. [79] reported declining disruption on citation-based measures of papers and patents. In life science, Azoulay et al. [2] observed increased contributions from outsiders after the premature deaths of eminent scientists. Our question is whether feedback from execution can help a proposer depart usefully from familiar ideas.

From search to learning. Automated research systems already couple proposals to tests. The AI Scientist [65] and the AI co-scientist [37] run agentic research loops; AlphaEvolve evolves programs against automated evaluators [75]; and Karpathy's autoresearch retains training changes that improve a measured objective [49]. Socher's *Eureka Machine* extends this vision to agents working with simulators and autonomous laboratories [105, 106]. These systems make judgment executable. Our question is what their evaluations can teach the system that proposes the next idea. Keeping a winning program preserves an output; training a proposer on those outcomes may preserve a capability that transfers to a new problem.

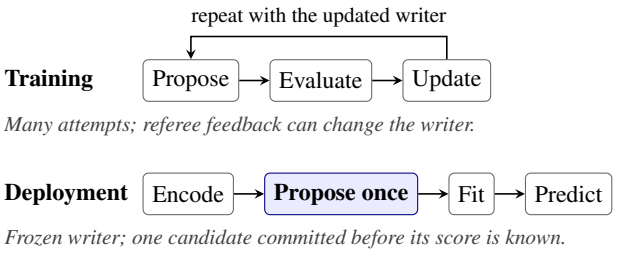


Figure 1: **Training feedback and the single-candidate contract.** Schematic. The writer updates during training; the emitted estimator fits during deployment. Whether the learned writer improves the first proposal is the empirical question in Section 7.

Key idea

Can judgment bought during training improve the first proposal at deployment?

The Zero move. AlphaGo learned from human expert games and searched heavily at every move [100]. AlphaGo Zero learned only from games against itself, and its network alone, with no search, rated 3,055 Elo. That is close to the 3,144 of AlphaGo Fan, the version that beat the European champion Fan Hui [100, 102]. This motivates an analogous goal for invention: learn from repeated program evaluations during training, then deploy a writer with a single-candidate budget, search = 1 (Box 1). The budget is a test condition, not a promise that every first proposal wins. Figure 1 separates the training feedback path from the frozen writer and fitting procedure at deployment.

Box 1: What "search = 1" means

- **One inference, one candidate.** For a new problem, the program writer emits one complete program ($B = 1$), with no retries, repairs, best-of-N, reranking or weight updates.
- **Everything is counted,** including reasoning tokens and problem encoding. One inference is one generation, not one forward pass.
- **Declare the fitting procedure.** Fitting the emitted algorithm to training rows is allowed. Any internal model or hyperparameter selection must be disclosed and counted in its compute budget. One outer proposal does not establish the absence of internal search. Training search is priced separately in Section 5.

A single candidate, with prior knowledge: human-designed program primitives and pretrained encoding components supply prior knowledge. The Zero analogy concerns the additional training feedback from the referee.

Engineered intelligence. Section 2 divides invention into four pillars: an aim, a source of variation, a means of judgment and a way to retain what survives. In EurekaZero each pillar is a specified component that can be inspected and tested. The aim is a declared task and metric. Variation comes from a grammar-constrained program writer. Judgment is a referee that scores each program on folds withheld from its fit. Retention is a pair of ratchets: a board's incumbent changes only on strict improvement, and a retrained writer replaces the serving one only after a held-out admission test (Table 1). We call this *engineered intelligence*: a capability whose aim, judge and memory are designed to be audited, rather than left implicit in a pretrained model. Pretrained components still supply prior knowledge (Box 1). Engineering fixes what the system is rewarded for and how its improvement is checked.

Contributions. We formulate algorithm invention as a game and design a learner for it, instantiated for tabular prediction tasks whose moves are programs in a typed language of learning primitives. The paper contributes:

1. **The anatomy of innovation:** four pillars (aim, variation, judgment and retention), with an idealized relation between proposal quality and the number of attempts (Section 2).
2. **The innovation game:** a board, move, referee and opponent, with a reward ordering and two ratchets, that make invention playable on many boards (Section 4).
3. **The economics of algorithmic self-play:** a cost model for training and for the cost per verified improvement (Section 5).
4. **EurekaZero:** a learner with a frozen problem encoder, a grammar-constrained program writer and a separate problem–program map, trained by expert iteration on referee scores, with safeguards against reward hacking (Section 6).
5. **An evaluation protocol for the single-proposal claim:** paired comparisons with frozen algorithms whose receipts record the model that designed them, performance curves against agents allowed repeated attempts, and a test of the map's ranking against a program-only control (Section 7).

What is and is not established. The paper separates three kinds of evidence. The game, the learner and the evaluation protocol are specified here. Retained admission and program-promotion logs from later adapter training describe development behavior (Section 6.8); they do not measure first-proposal quality on unseen problem families. The deployment comparisons of Section 7 are specified, but their outcomes are not yet reported, and we claim no advantage until they are.

Section 2 separates the roles in innovation. We then trace their AI building blocks (Section 3), define the game and learner, and specify the comparisons in Section 7. The outcomes will bound the claims discussed in Section 8.

2 The Anatomy of Innovation

We organize innovation into four roles: an aim, variation, judgment and retention. They connect a proposed program to its evaluation and to what the next attempt can learn. Producing

Table 1: **A four-part reading of traditions of discovery.** A dash marks a pillar outside a tradition's scope: Wallas analyzed a single act of thought. For Darwin and Campbell, the aim is a criterion set by the environment. Engineering is the design–build–test–learn cycle [74]; AlphaGo Zero follows Silver et al. [102]; EurekaZero is defined in Sections 4 and 6.

Tradition	Aim	Variation	Judgment	Retention
Darwin [25]	Fitness to the environment	Variation	Natural selection	Inheritance
Wallas [116]	Preparation	Incubation, illumination	Verification	—
Campbell [15]	A selection criterion	Blind variation	Selection	Retention
Popper [87]	The problem	Conjecture	Refutation	The surviving theory
The scientific method	Question	Hypothesis	Experiment	Publication
Engineering	Design target	Design and build	Test	Learn
AlphaGo Zero	Win the game	Moves chosen with search	Game outcome	Network weights
EurekaZero	Problem encoding and metric	Program writer	Referee on hidden folds	Incumbent pool and weights

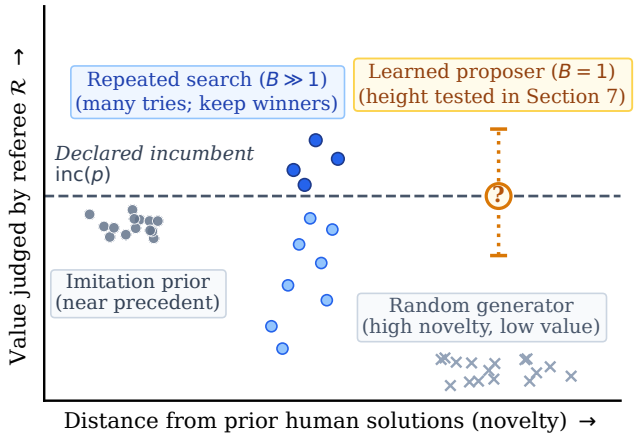


Figure 2: **Novelty is free; judgment is the inventor.** Schematic. Horizontal: distance from what people have already done. Vertical: value as judged by the referee. A random generator travels far and finds little. Imitation, repeated search and a learned proposer illustrate different ways to select candidates; their positions are conceptual, not measured limits. The value of EurekaZero's single proposal remains an empirical question for Section 7.

a new string is easy; finding a useful program is harder. We call judgment learned before the next attempt *taste*, and ask whether it can improve the first proposal.

2.1 Novelty is free

Novelty is the cheapest part of invention. Borges imagined a library holding every possible book of 410 pages [11], so every algorithm short enough to fit in a volume is already on its shelves. The library is useless, because nothing in it tells a reader which volume is true. A random string generator is the same library with a faster librarian: almost every string it emits is new, and almost none is worth running (Figure 2, lower right).

Language models show the same split. In a blinded study with 104 NLP researchers, ideas from a language model were rated more original than experts' ideas ($p < 0.05$) [98]. Once 43 experts had spent over 100 hours each carrying out ideas from both pools, the generated ideas lost significantly more ground on every metric [99]. Boden had predicted the asymmetry in 1998: machines, she wrote, would find generating new ideas easier than automating their evaluation [10]. These studies motivate a distinction between the novelty of a proposal and its value after execution; they do not establish that judgment is the only bottleneck in every domain.

2.2 The four pillars

Accounts of discovery from biology, psychology, philosophy and engineering provide a useful common vocabulary (Table 1). The four-pillar organization is our synthesis of these traditions, rather than a claim that their theories are identical.

Aim fixes the problem and what counts as better. Hamming asked colleagues, "What are the important problems of your field?" [40]. Pólya's first step is to understand the problem [85], and Popper argued that observation is always selective, so it presupposes a problem [87].

Variation proposes candidates. Peirce called abduction "the only logical operation which introduces any new idea" [81, CP 5.171]. Campbell modeled creative thought as blind variation [15, 104], and Koestler located creative acts in bisociation, the joining of two unrelated frames of thought [52]. Boden named three routes to a new idea: combining familiar ideas, exploring a conceptual space and transforming it [10].

Judgment tests a candidate against something outside the mind that proposed it: the attempted refutation [86], the experiment, peer review, and the referee of the common task framework, which scores every entry on data the competitors never see [29].

Retention preserves what survives: inheritance for Darwin [25], selective retention for Campbell [15], the textbook for a science and the incumbent for a game. It can store either an output or a way to propose better outputs. Archiving a winning program does the first; using evaluation feedback to update the generator attempts the second. This distinction connects the four pillars to the learning question in Section 1.

The traditions converge for a reason: Campbell argued that every genuine gain in knowledge needs a mechanism for introducing variation, a consistent selection process, and a mechanism for preserving and reproducing what is selected

[15, p. 381]. The aim is what keeps the selection consistent, which makes it the fourth pillar.

2.3 Taste is judgment moved upstream

Skill in invention is judgment that arrives before the try. For Poincaré, creating meant not making the useless combinations: "Invention is discernment, choice." The sterile combinations, he added, do not even present themselves to the inventor [83, pp. 324–325].

Chess masters show the same economy: in de Groot's think-aloud studies they considered about as many moves as weaker players but found the right ones [26]. Chase and Simon studied chunks through perception and recall tasks [18]. Much of that knowledge is tacit; experts know more than they can say [84].

AlphaGo's policy network learned to select moves from human expert games, then refined that judgment through self-play [100].

Taste has a price we can state. Suppose each try beats the incumbent with a fixed, nonzero probability q, independently of the others. The number of tries up to and including the first win is then geometric, with mean

$$\mathbb{E}[N_{\text{tries}}] = \frac{1}{q}, \tag{1}$$

where N_{tries} counts all candidate attempts, including incomplete or invalid outputs and failed executions. Under this fixed-probability, independent-try model, better proposal quality shortens the expected search.[1]

The equation isolates the value of learning before deployment. It does not rank random generators, pretrained models and agent systems without measuring their proposal quality. In particular, search $= 1$ fixes the number of candidates a system may submit; it does not require the probability of a win to approach certainty. A single-candidate system can fail, and every failure must remain in its reported results. Section 5 uses the same idealization to relate evaluation cost to the cost of a verified improvement.

Key idea

Novelty is free. Judgment is the inventor.

2.4 Learning from a referee

Taste depends on the feedback available to the learner. People learn much of it from the record of their field. Normal science, in Kuhn's account, solves puzzles inside a reigning paradigm [55]. The record also holds people in place: when a star life scientist dies prematurely, articles by outsiders to the star's collaborators flow into the field, draw on different literature and are disproportionately likely to be highly cited [2].

A model trained to predict text receives a signal about what was written, rather than a direct score for improving an algorithm. That does not prevent novel generalization. It motivates supplementing the historical record with feedback from execution, so a useful departure from precedent can be rewarded.

A referee offers such an additional teacher. AlphaGo Zero learned with no human data, from the rules and the outcomes of its own games, and found strategies outside traditional Go knowledge [102]. Its predecessor had already made such a move. As David Silver told Wired in March 2016, AlphaGo put the chance that a human would play its Move 37 against Lee Sedol at one in ten thousand, and it won that game [69]. The knowledge then flowed back to people. In more than 5.8 million moves by professionals from 1950 to 2021, previously unseen moves grew more frequent once superhuman programs arrived, and came with better decisions [97]. These results establish the possibility in Go. Whether an analogous signal improves algorithm invention must be measured in the innovation game.

2.5 Where the pillars live

Machines do not remove the pillars; they move them (Table 2). The most ambitious swarm design fits the same anatomy. In an October 2026 essay drawn from his book, Socher rests the Eureka Machine on "a living map of all human knowledge", "a unified model of physical reality", "high-fidelity digital twins" and "fully autonomous physical laboratories" [105, 106]. On top of them, millions of AI agents critique each other's ideas. In our terms, the map is retention; the model of reality, the twins and the laboratories are faster and better judgment; and the agents supply variation and critique.

Agent search and learning from a referee are complementary. A search can retain programs, revise its context or update its proposer; calling it a swarm does not determine which of these it does. The comparison in Table 2 therefore isolates a common deployment setting in which search leaves the proposer's weights fixed. EurekaZero instead aims to use training evaluations to improve the proposer before a new problem arrives. Both approaches still need a reliable referee, and the cost of training must be counted alongside any saving at deployment.

The four pillars describe the roles; the next section traces the methods that connect them.

3 The Road to Zero: Five Building Blocks

Earlier AI work supplies the building blocks for the four pillars of Section 2. Deep learning supports proposals; program evolution and reinforcement learning built judgment loops, and games made judgment cheap. The race for self-improvement now scales search to swarms of agents. We trace these lines of work to ask whether experience across algorithm-discovery tasks can improve the first complete program proposed for a new family. The combination matters more than a claim that any one component is new.[2]

[1] Adaptive search and correlated proposals need not follow Equation (1); independence alone does not say whether the approximation overstates or understates their cost. Empirical attempt curves are required for those systems.

[2] Each history keeps only the milestones the argument needs; Appendix A collects the referees those milestones rely on.

Table 2: **Where feedback can be retained.** The agent-search column describes fixed-weight deployment loops, not every agent system. The EurekaZero column states the proposed training and deployment protocol. In every column, people still set the aim.

	Human science	Agent search	EurekaZero
Aim	The researcher chooses	A person writes the goal	Problem encoding and metric, chosen by people for now
Variation	The scientist's ideas	Agents propose and critique	One candidate per test problem
Judgment	Experiments, peer review	Experiments, simulators, autonomous labs	The referee, on hidden folds
Retention	The literature	Logs, archives, a map of knowledge	Incumbent pool and the writer's weights
Where taste lives	Years of training, in one head	Weights fixed; context can adapt	Training on referee feedback
Deployment budget	Varies by problem	Multiple adaptive attempts	One candidate; success measured

3.1 Deep learning gave variation a voice

Deep learning built the variation pillar: machines that propose. Models learned representations from data [59]. AlexNet made the case on images [54], the Transformer modeled sequences with attention [113], and empirical scaling laws related language model loss to model size, dataset size and training compute [42, 48]. Sutton [109] drew the lesson: general methods that use computation win. Generative models then turned representations into proposals, from text to video [14] and code. Codex solved 28.8% of the HumanEval programming problems with one sample, and 70.2% with 100 samples, counting any that passed the unit tests [19]. That gap illustrates the value of a larger sampling budget. It is not a direct estimate of the geometric model in Equation (1): pass rates and the probability of beating an incumbent measure different outcomes.

Learned prediction and selection also support scientific discovery: protein structures that rival experiment in most CASP14 cases [47], 2.2 million crystal structures predicted stable relative to the Materials Project [68], and halicin, an antibiotic that then treated infections in mice [107]. CLIP put images and captions in one shared space [88], an analogy for our problem–program ranking map (Section 6). The reference map ranks scored programs within each problem; it does not condition or screen the writer. Ranking accuracy and proposal quality therefore need separate tests.

What EurekaZero takes: representations that read a problem, models that write code, and a separate ranking map.
The question carried forward: how to train a code proposer directly on verified algorithm quality, and test transfer beyond its training tasks.

3.2 Machines that invent algorithms built the variation–judgment loop

Program evolution joined variation to judgment: propose a program, score it, keep the better ones. Friedberg [35] let a machine vary its own instructions, Holland [43] made variation and selection a general method of adaptation, and Lenat's AM and EURISKO searched concepts and heuristics by measures of interest [63, 64]. EURISKO takes its name from the Greek verb behind *eureka*, as ours does. Koza [53] closed the loop with genetic programming: a population of programs, a fitness function as referee and selection as coach. A population retains useful programs; a learned proposer can also retain patterns that help construct programs for later tasks.

Later work sharpened both halves: superoptimization [67], program synthesis from examples [38] guided by neural networks [5], neural architecture search [125], and AutoML-Zero, which evolved learning algorithms from basic mathematical operations [89]. Then language models became the variation operator. FunSearch found cap-set constructions beyond the best known [91]: verified new solutions within a fixed task, bought with search. Lion, an optimizer found by program search, went into production at Google [20]. AlphaEvolve multiplied two 4×4 complex matrices with 48 scalar multiplications, improving on Strassen's algorithm in that setting after 56 years [75]. MLEvolve carries the loop into machine-learning engineering [30].

What EurekaZero takes: programs as moves, and the generate–judge loop.
The question carried forward: how much of a verified search can transfer into the proposer. DreamCoder already learns reusable libraries and a search-guiding model across tasks [31]; a single-candidate evaluation tests how useful a learned proposal is before additional task-specific search.

3.3 Reinforcement learning turned retention into taste

Reinforcement learning turns a referee's verdicts into a better player. Bellman [6] gave it an equation of value. Samuel's checkers program improved by playing a copy of itself [93], TD-Gammon reached near the best human backgammon players by self-play [110], and DQN learned Atari games from pixels [70]. AlphaGo learned from human games and searched at every move [100]; AlphaGo Zero learned by self-play alone [102]. AlphaZero mastered three games with one algorithm [103], MuZero learned a model for planning [95], and AlphaStar's league of mutually exploiting agents reached Grandmaster level in StarCraft II [114].

The key concept is expert iteration [1]: search is the teacher, the network is the student, and in the end the student alone plays strongly. AlphaGo Zero's raw network, with no search, nearly matched the version that had beaten a professional (Section 1); Box 2 shows how. Language models also learn from feedback: human preferences [22, 78] and rule-based checks of answers, as in DeepSeek-R1 [39].

What EurekaZero takes: the learning rule, and evidence from Go that a network can absorb search.

The question carried forward: how to define a reward for relative algorithm quality, then distinguish improvement on training games from improvement on unseen families.

Box 2: Self-play in 90 seconds

A player starts with only the rules and plays itself. A short search before each move finds better moves than instinct would, and the referee scores each finished game. The network learns to propose the moves the search chose (its policy) and to predict the winner (its value), so the next search is stronger. Over millions of games, search is distilled into intuition [1, 102]. *What if the board were a problem, and the move an invention?*

3.4 Gamification made judgment cheap

Games built the judgment pillar at scale. Shannon [96] chose chess because its moves and its goal are sharply defined. The common task framework made machine-learning research a game: shared data, enrolled competitors and a referee who scores every entry on a sealed test set [29], as in the Netflix Prize [7] and ImageNet [92]. People played science too. Foldit players beat automated methods on hard protein puzzles [23] and solved a protease structure that earlier attempts had not [50]; EteRNA players, whose designs were tested in a laboratory, found RNA design rules [60].

Then machines played problems. AlphaTensor's TensorGame beat the best known matrix-multiplication algorithms for several sizes [33], and AlphaDev's AssemblyGame found sorting routines now in the LLVM C++ library [66]. AlphaGeometry learned olympiad geometry without human demonstrations [111]. AlphaProof, using the Lean proof checker, and AlphaGeometry 2 jointly reached silver-medal standard on the 2024 International Mathematical Olympiad problems, using computation over multiple days [45].

What EurekaZero takes: the recipe for a game: rules, a referee, a score and a sealed test.
The question carried forward: which parts of a task can vary under a shared representation and referee. EurekaZero makes the problem an input to the proposer (Section 4); whether that supports transfer across families remains a result to establish.

3.5 The race for self-improvement scaled search

The race for self-improvement spans program search, agent scaffolds and learned policies. Good [36] foresaw an "intelligence explosion", and Schmidhuber [94] specified a Gödel machine that rewrites its own code once it proves the rewrite useful. STOP improves the scaffold that calls a language model, including itself, and its authors note: "Since the language models themselves are not altered, this is not full recursive self-improvement" [121]. The Darwin Gödel Machine's self-editing agents raised its SWE-bench score from 20.0% to 50.0% [122], and AlphaEvolve cut Gemini's training time by 1% [75].

Language models also play themselves. SPIN plays its previous iteration [21], Absolute Zero proposes and solves coding tasks under a code executor [123], and STP pairs a conjecturer with a Lean prover [28]. Bailey et al. [3] attribute plateaus in LLM self-play to reward hacking by the conjecturer and evaluate self-guidance in formal theorem proving. Our closest neighbors learn to discover. EvoTune fine-tunes on what its own evolutionary search finds [108], and TTT-Discover trains on the one problem at hand, at a few hundred dollars per problem [120]. Evolution Fine-Tuning learns from 156,000 search trajectories across 371 tasks [61], and DiscoRL meta-learned a reinforcement-learning rule that beat human-designed ones [77]. These approaches retain improvements in different places: a search trajectory, a task-specific learner or a reusable learning rule. Our comparison asks about the quality and cost of the first complete candidate, rather than treating these forms of learning as absent.

Research itself is becoming a swarm. The AI Scientist automates ideas, experiments and papers [65], and the AI co-scientist ranks agents' hypotheses in an Elo tournament [37]. Karpathy's autoresearch, released in March 2026, runs about 100 experiments overnight and keeps what improves [49]. In May 2026, Recursive raised $650 million to build AI that improves itself [13]. In the innovation game, an encoded research task would be the board and its evaluator the referee (Section 4).

Self-improvement comes in three strengths: a system can find better external algorithms, build better components of itself, or improve its own ability to improve, raising verified discoveries per unit of compute across generations. Each requires separate evidence (Section 8); a recorded training trajectory alone establishes none of them on an unseen task.

What EurekaZero takes: the destination, and the safety frame: verify every improvement before keeping it, and sandbox every program.
The question carried forward: whether learning across discovery episodes improves the first proposal, compared with the corresponding frozen proposer under the same deployment budget.

3.6 Where the roads meet

Table 3 distinguishes what a system produces from how it retains experience. TabPFN, MotherNet and KumoRFM provide neighboring examples of learning across tasks [34, 44, 73]. Their prediction or network-generation interfaces differ from the proposed readable-program output of EurekaZero, but a common candidate budget alone would not make their computation or task assumptions equivalent. Absolute Zero and STP learn by self-play, with correctness as feedback. Our proposed comparison measures improvement against a declared incumbent. That standard supplies the opponent; the next section specifies the board, program, referee and promotion rules needed to judge a move (Section 4).

Key idea

The test is whether experience on earlier discovery tasks improves the first complete proposal on a new one.

Table 3: **Related approaches retain experience in different forms.** This qualitative comparison separates output, deployment budget and retained state. Exact budgets depend on each study's protocol. The last row states EurekaZero's design; its proposal quality is still to be tested.

System	Output	Deployment budget	Retained experience	Feedback source
Genetic programming; AutoML-Zero	An algorithm	Study-specific search	Selected programs	Executable objective
AlphaTensor; AlphaDev	An algorithm for one family	Task-specific search	A policy for that game	Game reward
FunSearch; AlphaEvolve	An algorithm	Iterative search	Program archive	Automated evaluation
Research swarms (AI Scientist, autoresearch)	Experiments and code	Adaptive attempts	Context, logs and archives	Experimental outcomes
EvoTune; TTT-Discover; Evolution Fine-Tuning	An algorithm or construction	Protocol-specific search	Fine-tuned policy or search state	Search trajectories
TabPFN; MotherNet; KumoRFM	Predictions or network weights	Task-specific inference	Pretrained parameters	Training across tasks
Absolute Zero; STP	Answers and proofs	Decoding budget varies	Policy parameters	Correctness feedback
EurekaZero	An algorithm	One candidate	Proposed writer updates	Referee scores ranked within each problem

4 Innovation Is a Game

We define one attempt at algorithm invention as a game: the encoded problem is the board, a program is the move, the referee checks and scores it, and a declared incumbent is the opponent. Encoding the problem provides a shared interface; each task type still needs its own qualified encoding, language and evaluation protocol (Section 4.5).

4.1 Why games came first

Games became AI's laboratory because they supply four things at once: fixed rules, an impartial referee, a clear winner and cheap replay. Chess was chosen because it is "sharply defined both in allowed operations (the moves) and in the ultimate goal (checkmate)" [96], and checkers showed that a program could improve by playing itself [93]. Together these properties support repeated, reproducible evaluation. People scale judgment the same way, through prizes, leaderboards and the common task framework's "scoring referee" [29] (Section 3). Invention has the first three properties once the problem and the solution are written down, within the limits of Section 4.6; cheap replay is a question of cost (Section 5).

4.2 The rules

Table 4 sets the game beside Go. An illegal move is counted as a failed attempt. The reference writer reads the encoded problem; the referee keeps the incumbent as a separate comparison standard (Section 6.2). The incumbent's code and score are not part of that writer input.

Formally, the game is a tuple of seven parts,

$$\mathscr{G} = (\mathscr{P}, \phi_{\mathrm{P}}, \mathscr{S}, \phi_{\mathrm{S}}, \mathscr{R}, r, \mathscr{O}).$$

Here ϕ_{P} and ϕ_{S} embed a problem $p \in \mathscr{P}$ and a program $w \in \mathscr{S}$ as z_p and z_w. The referee $\mathscr{R}$ reports a program's validity and score, the reward r orders moves, and the opponent $\mathscr{O}$ supplies the incumbent $\mathrm{inc}(p)$. The tuple extends Rice's model of algorithm selection [90]: the player writes the algorithm instead of picking it, and a declared incumbent sets the comparison standard.

Table 4: **Go and the innovation game, element by element.** Moves differ most: Go's are cheap, while each program is costly to judge (Section 4.6). Go's game length is from Silver et al. [100].

	Go	The innovation game
Board	Stones on a 19×19 grid	The encoded problem
Move	Place one stone, or pass	Write one complete program
Legal move	Empty point; no suicide or forbidden repetition	Parses, type-checks, runs within limits
Illegal move	Not allowed	Stillborn: an automatic loss
Referee	The rules of Go	The verifier, on hidden folds
Win	Higher final score	Valid, and beats the incumbent
Opponent	A past self	The incumbent, often a past self
Rating	Elo	Win rate and Elo across problems
Moves per game	About 150, each cheap	One, costly to judge

A skeptic might ask whether a one-move game is a game at all. Formally it is a contextual bandit, with the board as context, the program as action and the verdict as reward [57]. It earns the name through an opponent that improves as the player does. The honest asymmetry is in the moves: Go offers about 250 legal moves per turn over about 150 turns, each free

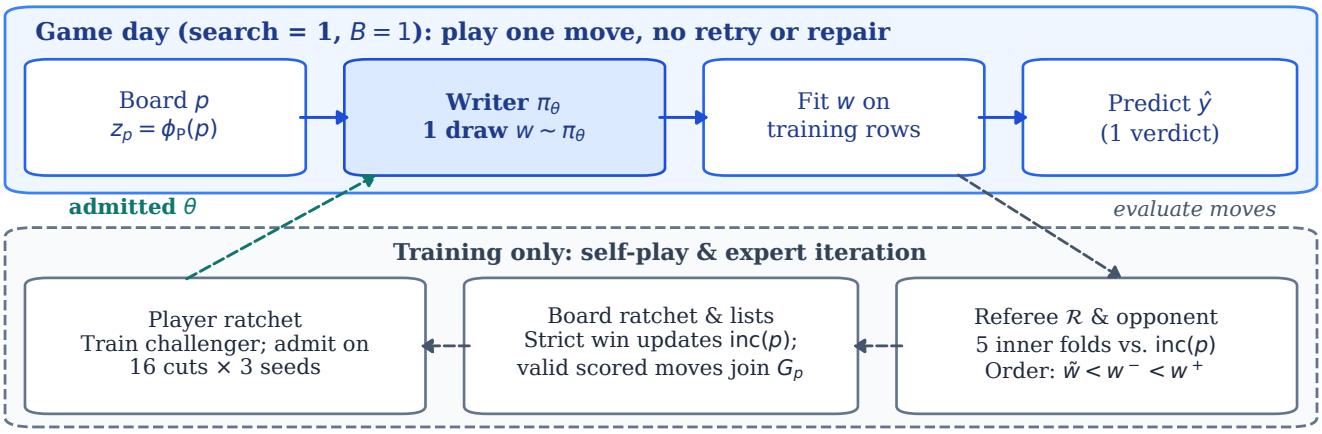


Figure 3: **Training plays the whole game; game day plays one move.** Schematic. Top row (blue): game day. Dashed box: training only. Outcomes are recorded; valid scored programs can enter the learning lists, while failed attempts still count against the budget.

to check [100], while this game offers one move, costly to judge. Under a single-attempt budget, the chance q of a win matters directly (Equation (1)). Whether training raises that chance on new families is the test in Section 7.

Figure 3 draws the loop. Game day plays only its top row, so nothing can send a move back for another try; that is search $= 1$.

4.3 The board and the move

The board is the aim pillar of Section 2, made machine-readable: a summary of the training data, the objective and metric, and the constraints. Landmark scores from cheap learners [82] and learned tabular summaries are possible encodings. The reference checkpoint in Section 6.2 uses a frozen 2,048-value summary with a 30-second cap. A useful board should preserve distinctions relevant to algorithm choice, remain stable under irrelevant reorderings, exclude final evaluation information and have a measured cost. These are requirements to test, not properties guaranteed by naming an encoder. People still choose the problem and define what counts as a useful solution.

The move is where invention gets its language: one program in a typed language over learning primitives. The reference language uses a 401-token vocabulary, with up to six transforms chosen from 31 and one of 22 prediction heads (Section 6.3). Types remove whole classes of stillborn programs before they run [71]. Canonicalization can merge syntactic variants; it need not identify every functionally equivalent implementation. AutoML-Zero reused scores when hashes of truncated predictions matched after a short training and validation probe [89]. Matching a probe does not establish agreement on other inputs (Section 6.3). The language bounds what can be expressed; changing its primitives changes that space [9]. The reference grammar also bounds composition depth. Structural constraints can prevent invalid programs; their effect on useful proposal quality must be measured (Section 7).

4.4 The referee and the opponent

The reward is an ordering first and a number second. Validity is a gate: any stillborn program scores below any valid one. Past the gate, programs rank by improvement over the incumbent on hidden folds. The reference promotion rule keeps the existing incumbent on a tie. Novelty never pays for losing: any novelty bonus (Section 5) must respect the order. For a stillborn program $\tilde{w}$, an also-ran w^- and a win w^+ on board p,

$$r(p,\tilde{w}) < r(p,w^-) < r(p,w^+).$$

The opponent is a ratcheting incumbent. In the reference procedure a program displaces a board's incumbent only by strict improvement on five inner folds under the same evaluation protocol. A retrained writer has a separate admission test on 16 held-out cuts with three seeds each (Section 6.5). These are distinct ratchets, as AlphaGo Zero's network admission was distinct from choosing a move during play [102]. Together the two ratchets make this self-play: the player keeps facing a standard often set by its own earlier play. Subtracting the incumbent's score does not change which move is best; what matters is what promotion changes. It raises the bar, turns yesterday's winners into today's also-rans and shifts which problems are worth playing. The learning targets themselves use each program's rank within its scored list (Sections 6.4 and 6.5).

4.5 One game, many boards

Binary and multiclass classification, regression, ranking, forecasting and relational prediction are prospective boards. Each needs a compatible encoding, program language, metric and validation protocol; a shared formal interface requires its own qualification for each task type. Beyond machine learning, examples of referees include sorting tests [66], evaluators for mathematical constructions [91] and accelerator-kernel timers [75]. These motivate extensions of the recipe, rather than results of this paper (Section 8).

Casting algorithm discovery as a game is not new, but earlier games wrote one family of problems into their rules, as

AlphaTensor did for matrix multiplication and AlphaDev for sorting [33, 66]. AlphaZero used one algorithm for three games and trained a separate network for each [103]. Here the problem is an input on the board, so one player can, in principle, learn many boards.

Key idea

A reliable referee makes a discipline a candidate for this game; transfer to that discipline still needs evidence.

4.6 The analogy's limits

A worked game, chosen by a rule fixed before inspection, is to be reported with the results (Section 7.6). The analogy breaks in four places. **The referee is noisy**: picking a winner by a noisy validation score biases that score upward [17]. Strict numerical improvement alone does not quantify confidence; independent evaluation and an uncertainty protocol are needed to interpret it. **Moves are expensive**, each a fit and a cross-validation (Section 5). **Judges can be gamed**, as METR documented for frontier models in June 2025 [115]; Section 6 lists our safeguards. And **no algorithm wins everywhere**: averaged uniformly over all problems, no learner has lower error off the training set than another, ours included [119]. Real problems are structured, though, and a player that reads the board can choose its bias board by board. Section 7 reserves the tests of that hypothesis against fixed frontier-designed algorithms, with paired outcomes beside Elo summaries [32]. Non-transitive comparisons are one reason not to reduce the evidence to a single rating [4].

Those rules define an attempt. Pricing a verified improvement must also count attempts that fail or lose.

5 The Price of Play

What would it cost to play the innovation game? A complete ledger counts encoding, input tokens, generation, repairs, referee execution, updates and operating costs. Replay cost constrains the game (Section 4), but a reproducible full-run price is still pending (Section 5.6). We separate planning assumptions from the measurements required in Section 7.

5.1 The bill in one line

For independent attempts with fixed win probability and cost distribution, expected cost per win is mean cost per attempt divided by that probability (Equation (1)):

$$C_{\text{win}} = \frac{C_{\text{attempt}}}{q}. \tag{2}$$

Here C_{attempt} is the mean incurred cost of one candidate attempt, including input processing, reasoning, generation, checks and referee execution. Work spent on failures remains in the mean. A repair that emits a revised candidate is another attempt; charge its work once. The q is the probability that one attempt beats the incumbent. It factors as $q = q_{\text{run}}\, q_{\text{beat}}$, the share of moves that run times the share of those that win. A move that runs and loses still costs money, so we divide by q, not by the survival rate. Generating a token costs about 2 FLOPs per active parameter, and training on one about 6 [48]. Cost projections require reproducible inputs and calculations (Section 5.6).

For an observed batch, cost per verified win is the total ledger divided by verified wins, including update and operating costs where applicable. This accounting does not require independent attempts. Adaptive search still needs empirical cost and attempt curves; a batch with no verified win has no finite observed cost per win.

The bill has two ledgers. Training pays for every attempted candidate, including failures; deployment pays for encoding, one proposal and its fitting procedure, with no repair. Fixed-weight agent loops instead buy evaluation during task-specific search; a trained proposer may amortize some of that work across later tasks. Whether the saving repays training requires a measured crossover, still pending in Section 7.

5.2 Wasted moves

An attempt can fail before or after it becomes a valid competitor. A stillborn attempt fails to produce a usable program under the declared protocol: generation is incomplete, parsing or type-checking fails, execution exceeds its limits or crashes, or outputs fail predeclared validity checks. An also-ran is valid but does not beat the incumbent. The pilot funnel planned for Section 7 (Figure 16, panel a) traces those stages and the compute spent before each failed attempt stops. If 90% of attempts fail to improve the incumbent, the independent-try model requires ten attempts per improvement in expectation. A typed language can exclude invalid structures before fitting (Section 4). Runtime, resource and output failures may consume part or all of the fit, and their incurred cost must be counted. Whether also-rans dominate is a pilot question. Learning, a changed search procedure or a changed opponent can affect the measured win rate; isolate the effect attributed to the writer.

5.3 The correction tax and the reasoning dial

This subsection and Section 5.6 concern language-model writers and the agent comparators of Sections 7.3 and 7.4; the reference network of Section 6, writer and map together, has 27.8 million parameters and writes at most 128 grammar tokens. Small models are cheap per token and can be dear per win. For generation alone, compute per token falls about 190-fold from a hypothetical dense 1.5T writer to an 8B one. With equal token counts and compute unit costs, the smaller writer can tolerate a win rate about 190 times lower before that component's cost per win matches. If it needs three times the tokens per attempt, the generation-only ratio falls to about 60. Repair turns also re-read a growing context. Encoding and referee costs further change the full-cost ratio, so the threshold for the complete bill in Equation (2) requires all components to be counted.

Reasoning is the second dial. Reasoning tokens may cut stillborn moves, but each is billed as output. The planned pilot

(Figure 16, panel b) compares cost per win over writer size and reasoning effort. A U-shaped curve is the hypothesis; the measurement decides where, and whether, the minimum sits.

5.4 Rewarding useful hallucination

A familiar solution can be a useful prior. Matching the incumbent's score earns a tie; promotion requires improvement under the same protocol. A *useful hallucination* here is an unfamiliar proposal whose value the referee verifies (Box 3). Possible exploration controls include temperature and entropy schedules, a curriculum past known solutions, and quality-diversity archives [62, 72]. Any novelty bonus must preserve the reward ordering in Section 4. These are design options. The reference procedure keeps up to 64 distinct valid policy draws at temperature 1, filling a short pool with grammar-uniform samples. Another 16 random exploration programs join that pool to seed verified search (Section 6.5). A measured exploration study must declare its controls and budget share and count unsuccessful attempts; its quality–cost trade-off belongs with the results (Section 7).

Box 3: What a referee verifies

An unfamiliar program is a conjecture; the referee checks whether it runs and improves the declared score. Verification establishes that bounded result. A claim of discovery also needs comparison with known methods (Section 8). AlphaGo's Move 37 was unlikely by its own estimate and won its game (Section 2.4). A referee does not make strange moves good. Its reliability and execution cost must be established for the task (Section 6).

5.5 How many games?

Three quantities differ by orders of magnitude. **Tries to find one improvement:** $1/q$ on average (Equation (1)). For independent tries with fixed $q = 1\%$, 69 attempts reach even odds and 299 reach a 95% probability of at least one win in that model. Report empirical attempt curves for adaptive or correlated proposals. **Experience to train a transferable player:** AlphaGo Zero generated 4.9 million self-play games in about three days [102], and AlphaZero played 44 million games of chess in about nine hours [101, 103]. An illustrative grid of 10,000 problems, 100 rounds and 8 attempts per problem per round gives 8 million attempts, the same order. The grid counts work, not learning: the number of problem families matters more, and pilot learning curves, read only within their measured range, must decide. **Precision to plan the test:** illustratively, 150 independent units at a win rate near 60% give an approximate 95% interval of ± 8 points. The actual study must compute uncertainty over independent families, rather than treating every dataset as an independent observation.

5.6 Pricing evidence still required

Mixture-of-experts writers use compute on active parameters but need memory for all parameters: Kimi K2 has 32B active of 1T, while DeepSeek-V3 has 37B active of 671B [27, 51]. Resident one-byte weights for 1T parameters alone require at least 13 H100 SXM GPUs with 80 GB each [76]; runtime state needs additional memory. This memory floor is separate from the cost estimates omitted below.

A cost projection needs a pinned model and hardware configuration, dated prices, measured or justified utilization, a reproducible calculation, and an explicit accounting boundary. Those inputs are not yet recorded, so this paper reports no cost projections; they remain a measurement task.

Key idea

Cost per verified improvement depends on both the cost of an attempt and the probability that it wins.

Any future projection of a fixed attempt budget must be converted to cost per win using a measured win rate and compared under a common task mix, incumbent standard and verification rule. Under Equation (2)'s fixed-probability model, doubling q halves expected cost per win only if mean cost per attempt stays fixed.

The reference checkpoint in Section 6 uses a compact network with 27.8 million parameters and at most 128 grammar-constrained tokens, with no repair in the proposed single-candidate deployment protocol. Its measured costs must be reported separately from any future scaling projection.

The method that follows aims to raise q through training. Whether it succeeds must be tested on one proposal under the stated deployment budget.

6 EurekaZero: Learning Taste

The reference procedure uses verified search to train a program writer and a separate problem–program map that ranks scored programs. Deployment permits one draw from the writer, with the map absent. We describe this historical reference first, then the later adapter-ranking record in Section 6.8. Ranking a pool and proposing a new program are different tasks. The writer's first-proposal behavior requires the matched comparison in Section 7.2.

6.1 The player in one picture

In Figure 4 a frozen encoder reads the board and the program writer π_θ writes one complete program, as AlphaGo's policy network proposed a move [100]. The problem–program map v supplies a separate ranking score. In this reference checkpoint it does not condition or screen the writer's output, and is absent from deployment. Ranking a search's output does not by itself train or steer the proposer; search $= 1$ rests on the writer's single draw.

6.2 Reading the board

The reference checkpoint uses a frozen encoder to turn the board of Section 4 into 2,048 numbers, $z_p = \phi_{\mathrm{P}}(p)$, in three blocks. The first summarizes a pretrained row encoder's embeddings across rows, independent of row order (512 values).

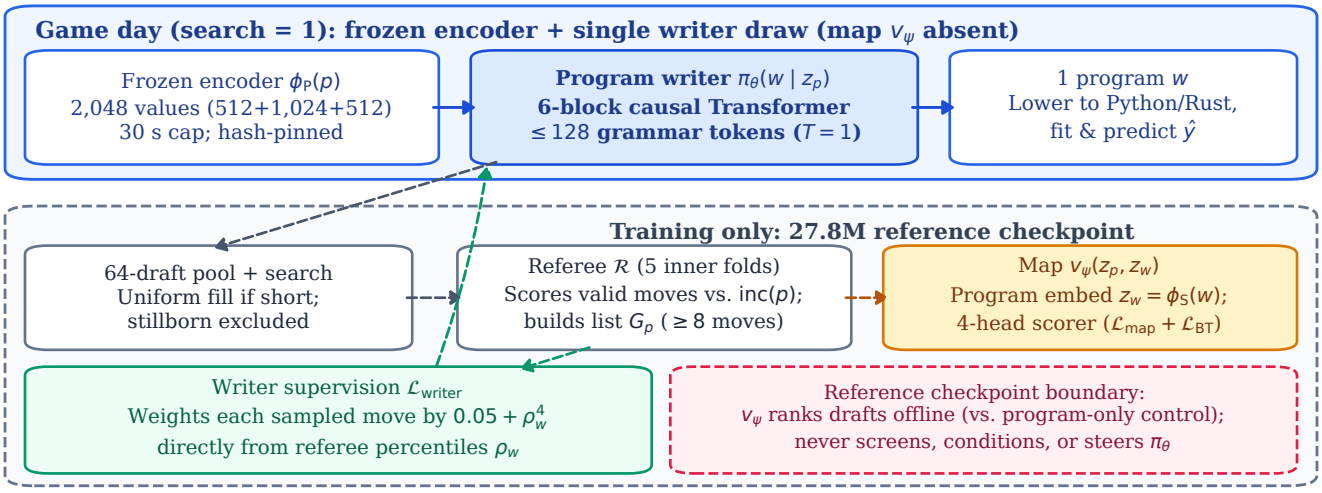


Figure 4: **A writer and a separate ranking map learn from scored programs.** Historical reference schematic. Top: game day, the whole of search = 1. Dashed box: training only. Search seeded with the writer's drafts proposes more programs, the referee scores them against the incumbent, and the scored lists retrain the writer (θ) and the map (ψ). Stillborn programs enter no list.

The second projects the hidden state of a pretrained third-party tabular regressor that reads the training rows and labels (1,024; Foundation, version `foundation_enc.v1.tabpfn`; license: proprietary, available on request; redistribution prohibited). The third passes the scaled targets through two frozen windows (512).

The encoder is fixed and pinned by hash in the reference procedure, rather than jointly trained with the writer. Encoding is bounded at 30 seconds per problem, and a context with any non-finite value is refused rather than imputed (Appendix C). The input contract permits training-side task evidence only; final held-out labels and their derived statistics must stay outside the input closure. A context's recorded declaration that no final target was used is part of the audit trail, not a substitute for checking that closure.

Whether problems with similar best programs sit together requires a separate representation study; Section 7.5 reserves that test.

6.3 Describing the move

A move is one program in the typed language of Section 4, spelled with a vocabulary of 401 tokens: up to six transforms from a set of 31, feeding one of 22 prediction heads. A grammar automaton masks every forbidden token at each step, the valid-only decoding of grammar-constrained generators [24, 56], so every finished program is grammatical. We credit that to the grammar, not to taste; taste must remove the moves that parse and then crash or lose. A program's identity is the SHA-256 hash of its canonical form. A separate hash of predictions on fixed training probes flags matching observed outputs. Those matches do not establish equivalence on other inputs. Without probe evidence, this behavior check reports novelty as unknown. The reference procedure calls for readable Python and Rust recipe exports that fit on the task's training rows. Reported winners also require the native–emitted parity check in Section 6.6.

6.4 One map for problems and programs

The map reads a program as $z_w = \phi_S(w)$, the mean of learned token embeddings. Pooling discards token order, limiting the map's ability to distinguish compositions built from the same tokens. It combines z_w with z_p three ways (as is, gated by the problem, and as a distance from a projection of it) and averages four heads into a score $v(z_p, z_w)$ with parameters ψ. It learns from lists: for each problem p with at least eight scored programs, G_p holds them and $\rho_w \in [0,1]$ is a program's percentile, 1 for the best. The map's softmax over the list is trained toward a target concentrated on the best [16]:

$$\mathcal{L}_{map}(p) = -\sum_{w \in G_p} \frac{e^{\rho_w/\tau}}{\sum_{u \in G_p} e^{\rho_u/\tau}} \log \frac{e^{v(z_p, z_w)}}{\sum_{u \in G_p} e^{v(z_p, z_u)}}, \qquad (3)$$

with $\tau = 0.25$. With a unique best program, the low-temperature limit concentrates the target on that program. This is a listwise ranking objective over programs scored on the same problem; the analogy to contrastive learning [88, 112] does not establish CLIP-style alignment across problems and programs. A Bradley–Terry term on the same lists sharpens the top [12]. Stillborn moves have no score, so this ranking objective covers only programs that run. A program-only head trained beside it tests whether the map reads the problem at all (Section 7).

Key idea

The reference game-day protocol omits the ranking map. Its single proposal tests what the writer has learned.

The writer and the map are separate branches, with separate context networks and token embeddings. In this checkpoint the map never touches the writer's draw. It ranks the draft pool, a ranking we score against the referee, but drops none, and it does not condition, screen or steer the writer. The writer's

learning targets come from referee scores directly. Thus ranking agreement with the referee, including comparison with the program-only control, can assess the map; attributing an improvement in proposals to the map would require an implemented influence path and a matched ablation (Section 7.5).

6.5 The self-play loop

The historical reference uses expert iteration [1]: verified search is the teacher and the writer is the student (Algorithm 1). The writer imitates scored programs sampled from the lists G_p, weighted by their rank among the referee's scores, inside a combined objective (Appendix C):

$$\mathscr{L}_{\text{writer}}(p) = -\sum_{w \in G_p} \left(0.05 + \rho_w^4\right) \log \pi_\theta(w \mid z_p), \tag{4}$$

$$\mathscr{L} = \mathscr{L}_{\text{writer}} + 0.1\,\mathscr{L}_{\text{map}} + 0.25\,\mathscr{L}_{\text{BT}} + 0.1\,\mathscr{L}_{\text{runtime}} + 0.05\,\mathscr{L}_{\text{prog}}. \tag{5}$$

The writer loss in Equation (4) is schematic: it displays the rank weights. The native implementation also smooths grammar-legal targets, averages token losses within each program, and normalizes by the sum of program weights (Appendix C).

Here $\mathscr{L}_{\text{BT}}$ is a Bradley–Terry loss on the same lists, $\mathscr{L}_{\text{runtime}}$ a Huber loss for a head that estimates runtime cost, and $\mathscr{L}_{\text{prog}}$ the Bradley–Terry loss of the program-only control head. Losing moves still teach: a median-ranked program carries about a tenth of the top-ranked program's weight in the writer loss. Its referee score also contributes to the map's ranking targets.

Two ratchets make this self-play rather than imitation of a fixed archive (Section 4). A program displaces a board's incumbent only by strictly beating it. A retrained writer takes the held-out admission test: 16 cuts with three seeds each. It replaces the serving writer only if its mean log error ratio favors it, it wins at least as many cuts as it loses, and it fails no more often. Here a cut is a held-out evaluation case fixed before the admission test. For each cut, the rule takes the natural log of paired best-error ratios (challenger divided by serving) and averages over matched seeds; it then averages across cuts. Negative values favor the challenger. This admission comparison is separate from the single-program comparison in Section 7.2.

The historical reference. The writer is a decoder-only transformer (six blocks, width 512) that reads the context as a prefix token and writes at most 128 tokens; with the map, the network has 27.8 million parameters. The supplied source notes describe training on eight A100 GPUs in September 2026, with a corpus of about 10.2 million scored programs from 80 training datasets. This paper does not document the evidence on which the checkpoint was accepted; that record and a pinned runtime are required inputs to the evaluation contract (Section 7.1). These specifications identify the historical reference; they do not describe the later adapter snapshot or establish its deployment performance. Details belong in Appendix C.

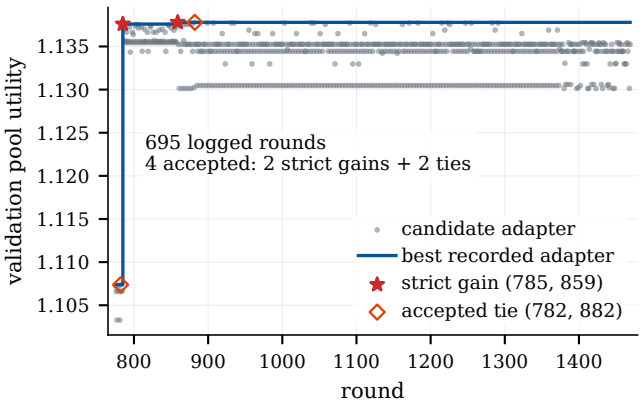


Figure 5: **Adapter validation and admissions.** Grey: candidate utility. Blue: recorded best. Stars: strict best increases; hollow diamonds: accepted ties. The annotation counts rounds in this window.

6.6 Keeping the referee honest

A learner rewarded by a referee can exploit its flaws. In February 2025 Sakana AI walked back claims after its AI CUDA Engineer exploited the evaluation code [117]. In June 2025 METR reported frontier models patching their scorers [115], and 16% of 1,968 agent-benchmark tasks can be passed without being solved [124]. We require the evaluation protocol to withstand such attacks. Selection, promotion and final scoring must use separate data (inner folds, held-out cuts, a final fold scored once), with sealed meta-test families outside the learner's inputs. The proposed comparisons require sandboxed execution using seccomp and Landlock, with memory, swap and time limits. A planted label leak and a sealed canary must both be caught, and every reported winner must be re-run from its emitted code, with native and emitted predictions agreeing within $10^{-9}\max(1, |\hat{y}|)$. One earlier leaderboard entry for our search selected each dataset's best test-scored outcome from a ledger of 63,108 evaluations; we exclude it as test selection (Appendix B).

6.7 Game day: search = 1

The proposed protocol in Algorithm 2 permits one encoder pass, one draw of at most 128 tokens at temperature 1 from a recorded seed, lowering to code, and the fit. The map is never run, and none of training's exploration programs is added. A draft that does not finish, crashes or times out is stillborn and scores as a loss; it is not replaced. Reported costs count the encoder and writer with the fit, and any later search is reported apart, as Silver et al. [102] reported AlphaGo Zero's raw network apart from its search (Section 7).

The deployment budget is a protocol constraint. Whether training improves the quality of that single proposal is a separate empirical question for the matched comparison in Section 7.2.

6.8 Where training stands

The retained CSVs describe a later adapter-training snapshot, separate from the historical reference above. An adapter is the

Algorithm 1 The reference self-play loop (training).

Require: training problems; serving writer π_θ and map v; incumbents $\mathrm{inc}(p)$

for each round **do**
  for each sampled training problem p **do**
    $z_p \leftarrow \phi_{\mathrm{P}}(p)$; keep the first 64 distinct valid draws from $\pi_\theta(\cdot \mid z_p)$ ▷ $T = 1$; grammar-uniform fill if short
    verified search, seeded with the drafts and 16 random exploration programs, proposes more programs ▷ the teacher
    $\mathscr{R}$ fits and scores each distinct program on 5 inner folds ▷ stillborn: no score
    if a program strictly beats $\mathrm{inc}(p)$ **then** it becomes $\mathrm{inc}(p)$ ▷ board ratchet
    add every scored program to the list G_p ▷ winners and also-rans
  train a challenger (θ', ψ') on the lists
  if it beats π_θ on held-out cuts **then** $(\theta, \psi) \leftarrow (\theta', \psi')$ ▷ player ratchet

Algorithm 2 Proposed game-day protocol (search = 1).

Require: new problem p with training rows and metric; frozen ϕ_{P} and π_θ

$z_p \leftarrow \phi_{\mathrm{P}}(p)$ ▷ read the board
$w \sim \pi_\theta(\cdot \mid z_p)$, one draw at $T = 1$ ▷ one move; no retry
$f \leftarrow \textsc{Fit}(w,$ training rows$)$ ▷ as any algorithm is fit
$\hat{y} \leftarrow f($new rows$)$ ▷ a stillborn w scores as a loss
return w, $\hat{y}$

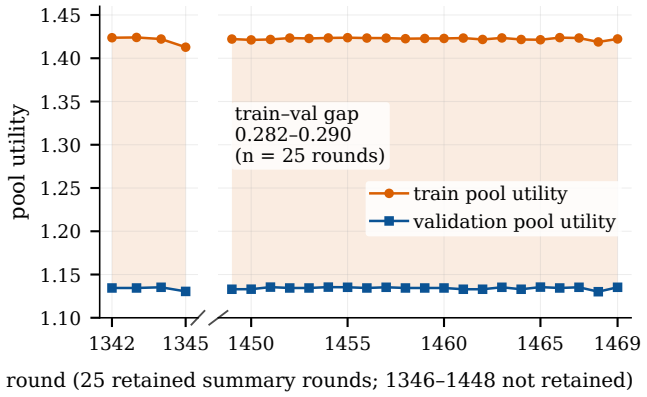


Figure 6: **Train and validation aggregates on retained summaries.** The 25 observations span rounds 1342–1469; a broken axis separates the observed blocks. Missing rounds are unobserved, and the reported utility gap covers only these summaries.

trainable adjustment evaluated by this admission loop. Utility is the logged scalar score, with larger values treated as better. The admission CSV records rounds, candidate and best validation utilities, acceptance flags and timestamps. The tables omit the adapter architecture, score normalization and evaluation population, so they do not support comparisons with other models. The source inspection that follows describes the loop implementation. It does not yet bind that source or the adapter's identity to the recorded run. Logged utilities alone do not identify the snapshot's relationship to the historical writer or map.

In the inspected loop implementation, heuristic program edits, retrieval and grammar sampling supply candidate programs. Only an adapter that ranks proposals by predicted score is optimized; the encoder and other modules stay frozen. This loop has no neural program writer. Its pool-utility calculation uses the recorded scores and gate outcomes of the first four candidates as revealed history, then selects the candidate with the lowest predicted score from the pool, including those entries. This is a conditional ranking task, separate from a single new program drawn by the historical writer. Training puts the lowest recorded scores first in its slates; utility evaluation uses stored pool order. The two calls can therefore reveal different examples.

The inspected caller supplies modality tokens and a mask while zeroing separate context and prior inputs. Its objective combines scalar evaluation-score regression and listwise matching, with a fixed target for failed gates. The reporting code averages the negative selected scores, assigning a fixed penalty to failed gates.

Candidate programs are evaluated on two training-side folds; the historical reference procedure specifies five. Evaluations are logged separately from training-pool admission: ordinary admission retains only programs that pass the gates and strictly improve the incumbent for their training context. Historical programs have a separate admission path. The adapter's default gate can accept equal utility; rejection restores model and optimizer states.

[TBD: Bind the retained snapshot to the exact loop source, adapter, candidate pools, revealed-history inputs and utility normalization.]

The admission log contains 695 consecutive rounds. Four rows are marked accepted, but only two strictly increase the recorded best validation utility within this window. Its final recorded best-validation value is 1.1378, with no later increase in the retained interval (Figure 5). Acceptance flags and improvements in this value are therefore distinct counts. Comparing these utilities across harnesses requires matched data, normalization and scoring protocols.

The more detailed summaries cover only 25 of those rounds. In that subset, training utility exceeds validation utility by 0.282–0.290 (Figure 6). This is a difference between reported aggregates on separate training and validation contexts; it does not isolate a cause or establish behavior in the missing summaries. The late validation plateau is evidence to explain, rather than proof of useful transfer. Encoder integrity and parameter updates require their own receipts; the plotted utility tables do not establish either. Figure 7 shows the loop's operating measurements from the same snapshot.

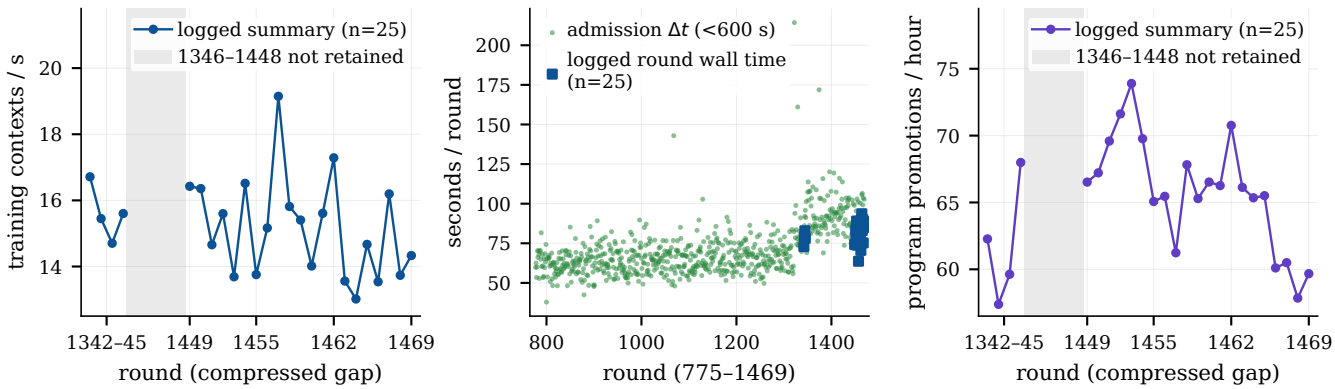


Figure 7: **Operating measurements from the retained snapshot.** Left: distinct training contexts divided by completed-round wall time. Middle: filtered time between admission timestamps (green) and logged round wall time (blue squares); longer timestamp intervals are omitted as indicated in the legend. Right: program promotions per hour, computed from strict improvements to training-program incumbents and accumulated round wall time over up to twenty completed rounds in the reporting process, including qualification waits. Program promotions and adapter acceptances count different events. These measurements describe the training loop, rather than the accuracy or cost of one-proposal deployment.

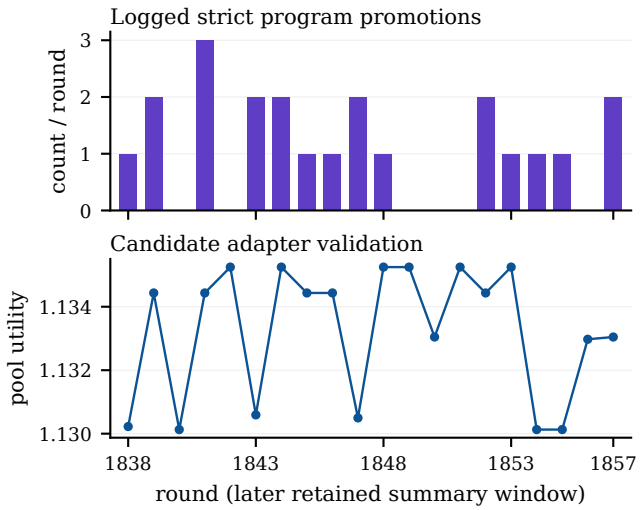


Figure 8: **Program-incumbent events and validation summaries.** Top: logged strict improvements to training-program incumbents per round. Bottom: candidate adapter validation pool utility. Both panels cover the same later retained summary window. These are development measurements; independent first-proposal comparisons remain required.

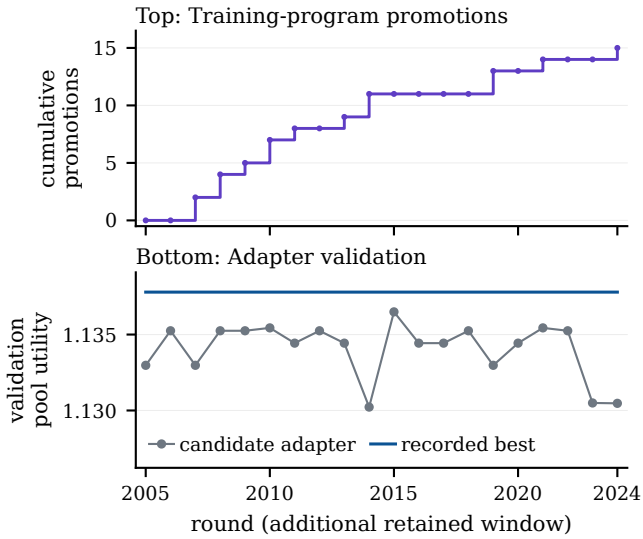


Figure 9: **Program promotions and adapter utility in an additional window.** Top: cumulative logged strict improvements to training-program incumbents, reset to zero at the start of this window. Bottom: candidate adapter validation utility and the recorded best. Both panels use the same 20 consecutive summary rounds. These are development measurements; neither the event count nor the utility curve establishes first-proposal improvement.

A separate later log slice aligns journaled training-program promotions with candidate validation summaries in Figure 8. These events record numerical improvements to training-program incumbents; the journal supplies no confidence interval for them. Program promotions and neural adapter admissions remain distinct events. The later slice is shown separately from the earlier trajectory.

A further development window appears in Figure 9. The cumulative count starts from zero for this window and includes only journaled strict program promotions. The adapter panel separates candidate validation utility from the recorded best; progress in the program pool and admission of a new adapter are different events. Figure 10 shows a separate subsequent window.

Historical GP target scores provide a separate view in Figure 11. The 159 repeated dataset/cut/arm groups cover 55 distinct dataset/cut pairs; the 64 groups with lower final scores cover 23 pairs. Shared incumbent and teacher seeds make the arms dependent. These retained development outcomes do not establish that learning improved the proposal policy.

Figure 12 follows one historical dataset/cut, chosen after inspection for the largest policy-seeded relative improvement in the preceding comparison set. Every reported generation

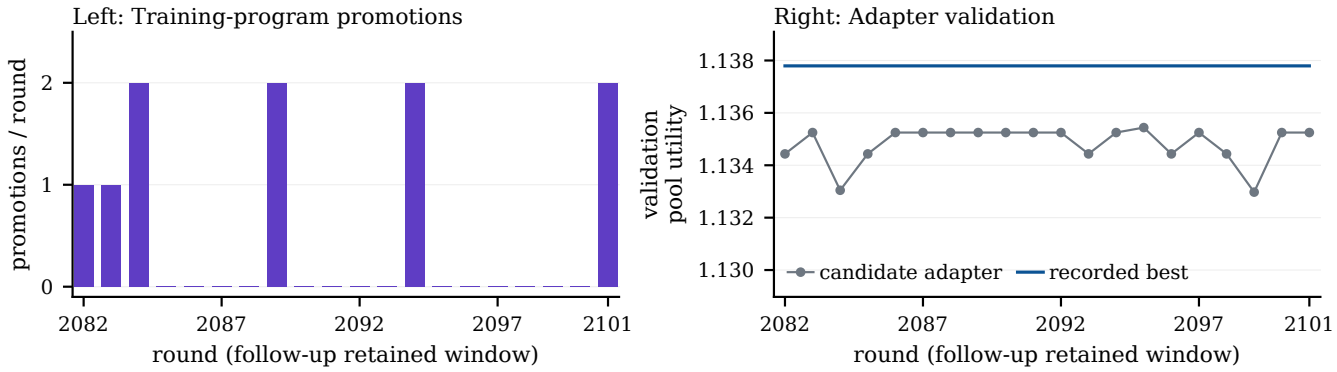


Figure 10: **Program events and adapter validation in a follow-up window.** Left: journaled strict training-program incumbent improvements per round. Right: candidate adapter validation pool utility and the recorded best. Both panels cover the same 20 consecutive summary rounds. The summaries mark no adapter admission. Independent first-proposal and transfer tests remain required.

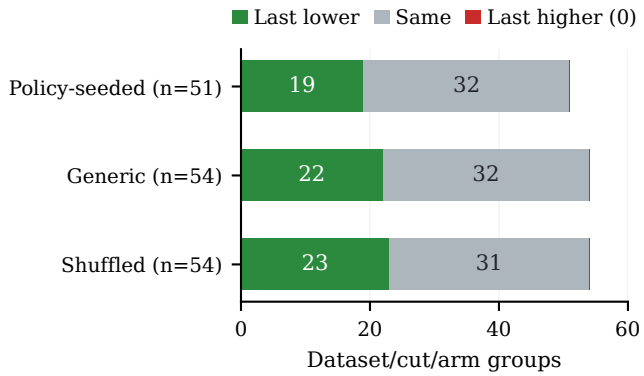


Figure 11: **Historical GP target progress.** First-to-last reported best development RMSE at 1,000 evaluations per search, grouped by search arm. Lower is better; unchanged scores are included. The arms shared incumbent and teacher seeds, and elapsed time varied. These counts do not measure independent discoveries or a learned-policy advantage.

is shown, without applying a cumulative minimum. All three arms finish at the same value; the policy-seeded arm first records it one generation after the controls. Shared seeds prevent treating their matching outcomes as independent discoveries.

For the adapter record in Figure 5, Figure 13 summarizes candidate gaps to each round's recorded best. Each retained round contributes once; the curve does not count independent tasks or distinct programs.

Program-incumbent gains, adapter admissions and historical target scores measure different outcomes. Section 7 specifies the independent comparisons needed to test whether training improves the writer's first proposal on new families.

7 Results

This section specifies the independent comparisons needed to test whether training improves the first proposal for an unseen problem. The development records in Section 6.8 track program search and adapter admission; their scores inform those decisions. They cannot settle the deployment question. Table 5 summarizes the controls and status of the pending comparisons. **[TBD: Add independent evaluation evidence before making a performance claim.] [TBD: Supply a source-bound benchmark before claiming deployment throughput.]**

7.1 Evaluation contract

Freeze the writer, permitted task inputs, output support, decoding rule and budgets before opening final outcomes. Pin dataset versions and content hashes, row splits and family assignments. Separate unseen problem families from training and development, and report which task types were actually tested. Report known training and pretraining exposure for the writer, encoders and learned comparators, with unresolved provenance marked explicitly. Bound unseen-family claims to the exclusions actually established. Each EurekaZero submission follows Algorithm 2; failed generations and executions remain in the denominator. Report paired results and uncertainty at the family level, alongside encoding, generation, fitting and prediction costs. Disclose any internal model or hyperparameter selection in the emitted procedure, even when the writer submits only one outer candidate. **[TBD: Document the final problem inventory, dataset identities, row splits, family assignments and exclusions, component provenance, evaluator, checkpoint, seeds, task metrics and failure accounting.]**

7.2 The effect of training

Test the learning claim with a frozen reference writer as the control. Predeclare that checkpoint and the evaluated training intervention before opening final outcomes. Hold the encoder, program grammar and output support, permitted task inputs, decoding rules, fitting protocol, referee and resource budgets constant; vary the writer checkpoint. Changes to the support, encoder or evaluator belong in separate conditions.

Pair the first draws under predeclared seeds and report all of them, with no selection of the best seed or candidate. Report paired family-level outcomes, failures and costs

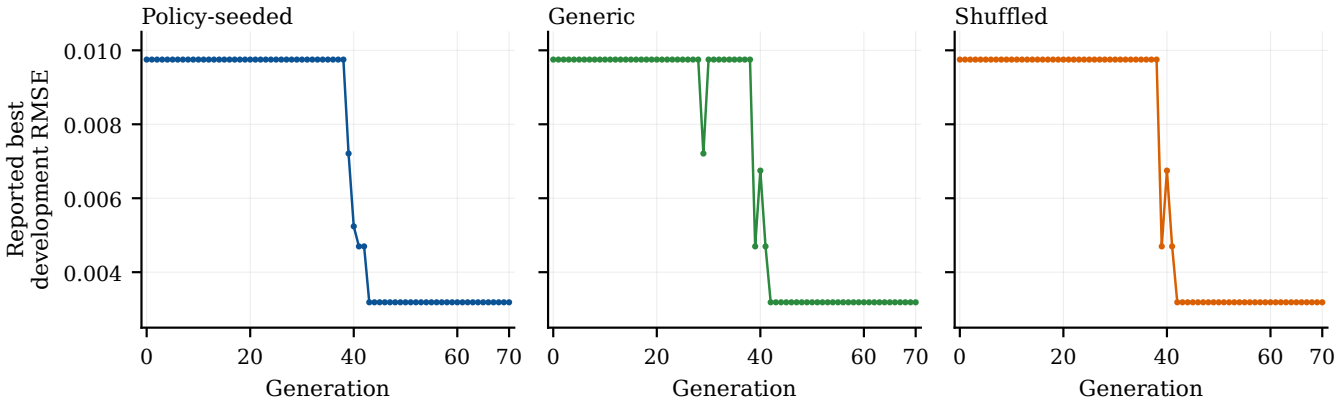


Figure 12: **Post-hoc GP target trajectory.** All 71 generations per arm for one historical dataset/cut, at 1,000 evaluations per search. RMSE is in historical evaluator units; lower is better. The arms shared incumbent and teacher seeds, and elapsed time varied. This selected development example does not establish a learned-policy advantage.

Table 5: **Evaluation plan at a glance.** Each test is specified in the subsection named in the first column. No outcome is reported yet; the contract in Section 7.1 fixes inputs, budgets and splits before outcomes are opened.

Test	Question	Comparison or control	Reported in	Status
Effect of training (Section 7.2)	Does training improve the first proposal on unseen families?	A frozen reference writer under the same budget, paired first draws	Text of Section 7.2	Pending
One proposal against fixed methods (Section 7.3)	Does one task-conditioned program compete with methods that run unchanged across tasks?	Frozen algorithms whose receipts record the designing model	Table 6 and Figure 14	Pending
The cost of more attempts (Section 7.4)	What do successive attempts buy, per attempt and per unit of cost?	Agents allowed successive attempts; the pilot cost funnel	Figures 15 and 16	Pending
What the map contributes (Section 7.5)	Does the map read the problem, and does it change the writer?	The program-only head; a matched causal ablation	Table 7	Pending
Failures and a worked example (Section 7.6)	How do attempts fail, and what does one game look like?	Terminal outcome classes; a rule-selected example	Table 8	Pending

alongside source-bound evidence of optimizer updates and parameter changes. **[TBD: Supply the matched first-proposal comparison and training-update evidence before claiming transfer.]**

7.3 One proposal against fixed methods

The proposed comparison asks whether one task-conditioned program competes with methods that run unchanged across tasks. The comparators are frozen algorithms whose receipts record the model that designed them: one designed by `claude-opus-5-5` and one by `gpt-6-astra-wm`, the latter identifier as reported by its authoring session. A comparator without a recorded model identity is excluded rather than guessed. Record prompts, dates and implementation checks, and compare under the same task metrics. Table 6 reserves the comparison; no winner is assumed.

7.4 The cost of more attempts

Compare the single proposal with agents allowed successive attempts and development feedback. Keep final outcomes unavailable to those agents. Count every candidate attempt, including incomplete or invalid outputs and failed executions; a repair that emits a revised candidate is another attempt. Charge all task-time computation. Plot performance against both attempts and cost; if an agent does not reach a reference performance within its budget, report that limit. **[TBD: Supply attempt curves, matched budgets, repeated seeds and uncertainty.]** The pilot cost measurements planned in Sections 5.2 and 5.3 are planned for Figure 16.

Table 6: **Planned paired comparisons.** No outcomes are yet supplied. Report ties, failures and excluded cases alongside wins.

Comparator	Outcome and uncertainty
Designed by `claude-opus-5-5`	**[TBD: paired evaluation]**
Designed by `gpt-6-astra-wm`	**[TBD: paired evaluation]**

7.5 What the map contributes

A representation study should test whether nearby problem encodings share useful programs, with data disjoint from fitting the map. Ranking agreement should also be compared with the program-only head, so a global preference

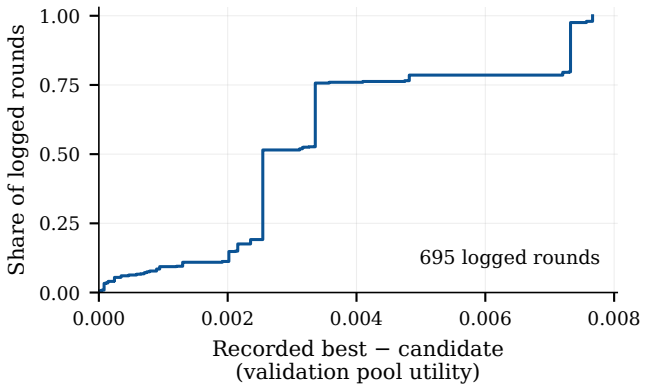


Figure 13: **Candidate gaps to the recorded best.** Cumulative share of the same 695 admission rounds as Figure 5, against recorded best minus candidate validation utility. The best is logged after admission, so zero includes strict gains and ties. Repeated adaptive rounds are not independent trials.

[TBD: Leaderboard: supply the evaluated methods, paired outcomes, rating protocol and uncertainty.]

Figure 14: **Leaderboard pending.** Elo is a summary of a specified comparison population, not a substitute for paired task results.

[TBD: Search curve: performance versus attempts and compute, including failed attempts and unreached targets.]

Figure 15: **Deployment search comparison pending.** The single-candidate condition fixes a budget; its measured quality remains to be established.

for common programs is distinguishable from reading the problem. **[TBD: Supply problem-similarity and program-only-control evaluations.]**

A ranking head can agree with a referee without improving the writer. In the checkpoint described in Section 6.4, the map neither conditions nor screens the writer's draw. Its ranking accuracy therefore cannot establish a reduction in wasted proposals. A causal ablation needs a declared route by which the map changes training, with the writer, training budget and deployment protocol otherwise matched. **[TBD: Identify an implemented influence path or limit the study to ranking.]**

7.6 Failure cases and a worked example

Describe unsuccessful proposals and coverage limits as well as wins. Assign each attempt one terminal status: incomplete generation, parse or type-checking refusal, execution or resource failure, invalid output under declared checks, and valid ties, losses and wins. Every attempted candidate contributes its incurred cost. Unresolved outcomes retain an explicit unknown status; fix their treatment and the validity rules before opening final outcomes. **[TBD: Supply attempt inventories and costs by outcome class, identifying unresolved cases.]**

Choose the worked example by a rule fixed before inspection, and show its emitted program, fitting procedure and comparison outcome; if the selected category is empty, report that fact. **[TBD: Supply the example selection rule, actual program and verified outcomes, with no replacement of failed attempts.]** Table 8 lists the record for the selected game. The attempt permits no repair or second try, and its sealed verdict is reported without updating the writer.

[TBD: Pilot cost measurements: supply per-stage failure shares and cost per verified win.]

Figure 16: **Pilot cost measurements pending.** Planned panels: (a) the share of development proposals that run, score, beat the incumbent and survive re-verification, with the compute spent at each stage; (b) cost per verified win by writer size and reasoning effort. **[TBD: Supply pilot data, seeds and family-level uncertainty; identify settings with no verified win without assigning a finite cost per win.]**

Table 7: **Map evidence still required.** Ranking quality and a change in first-proposal quality are separate outcomes.

Question	Required evidence
Does the map rank candidates?	**[TBD: held-out ranking study]**
Does it improve the writer?	**[TBD: matched causal ablation]**
Does it reduce execution failures?	**[TBD: failure-class comparison]**

Table 8: **Worked-game record.** Fields reported for the game selected by the rule above. An empty selected outcome category is reported rather than replaced.

Field	Record
Problem, task and metric	**[TBD: Identify the problem, task and metric.]**
Encoding time	**[TBD: time]**
Emitted output	**[TBD: program or unfinished draft and size]**
Check and fitting outcomes	**[TBD: outcomes and time]**
Hidden-fold score of a valid fit	**[TBD: score]**
Incumbent's score	**[TBD: score]**
If no valid fit	The failure and its incurred cost, in place of the scores

7.7 Earlier development diagnostics

The supplied method notes report 3,227 exact ties in 3,268 paired policy-seeded versus generic final searches, plus a separate audit in which 98.84% of reported winners were already in their starting seed populations. The paired searches shared seeds, repeated development contexts and had unequal wall times; they were not independent first-proposal draws. The two historical inventories differ.

For a separate earlier model evaluated on development contexts, the notes report fresh-program pairwise ranking accuracy of 0.506 with an interval of [0.441, 0.568], which includes

chance. The reference checkpoint and later adapter snapshot in Section 6 require their own ranking evidence. These observations describe historical search and ranking behavior; the matched first-proposal test above is still required. **[TBD: Pin the inventories, checkpoints and seeding, timing and ranking protocols, separately from the final comparisons above.]**

The independent comparisons specified above will determine which claims Section 8 can support.

8 Discussion

The comparisons in Section 7 will decide whether training improves first proposals on independent new families. This section separates the claims that each outcome would support. The development record alone cannot establish transfer, useful new composition or autonomous scientific discovery. **[TBD: Reconcile every conclusion below with the completed evaluation.]**

8.1 Learning to select and learning to invent

A better task-dependent choice from a fixed program archive demonstrates selection. A useful composition absent from the training archive is a stronger result, and requires an output space that permits it. Token-by-token decoding can produce either, so report the admissible output space, the training archive and the relation of each proposed program to that archive. Absence from an archive establishes novelty relative to that archive, rather than to the scientific literature. Different program spellings can also express the same computation. Claims of invention therefore need comparisons of program behavior and known methods, alongside predictive quality. The evaluated checkpoint's outputs, compared with its training archive, decide which of the two claims applies.

8.2 Moving judgment across problems

If learning improved first proposals on new families, that would support the hypothesis that training evaluations can be retained as a transferable proposal capability. It would not show that one proposal wins on every problem. Test-time search may improve the program returned for a task; its value should be weighed against the cost of training a better writer. The search curves and full cost account in Section 7.4 bound this tradeoff.

8.3 The referee defines the boundary

The framework optimizes what its referee measures. Repeated development comparisons can overfit that referee, while a restricted language can hide useful algorithms from the learner. An admission based on a higher development score records a selection decision. It does not quantify uncertainty or guarantee improvement in the target population. The independent evaluation must keep the chosen checkpoint fixed and report paired uncertainty, including failures and ties. Task representations, training families and runtime limits further bound the result. A referee for a new scientific domain would need its own validity and cost evidence; analogy to a game supplies neither. Observed failures, distribution shifts and the remaining tests of the referee belong with that evaluation (Section 7.6).

8.4 A bounded claim about self-improvement

Improving an external algorithm, improving the proposer and improving the rate at which the proposer learns are distinct claims. Evidence for the first does not establish the others. An optimizer trace with verified parameter changes can establish that training occurred; a lower training objective measures the fitting criterion. Neither alone shows that first proposals improved on unseen families. People still choose the task, metric, representation and resource limits. Any claim must stay within the level supported by Section 7; stronger claims require further evidence.

8.5 Limitations

Beyond the pending evaluation, four limits belong to the design itself. The reference boards are tabular prediction tasks; other task types need their own qualified encoding, language and protocol (Section 4.5). The program language bounds what can be proposed, and its primitives encode human prior knowledge (Section 4.3). The referee is noisy and can be gamed, and strict improvement on inner folds does not quantify confidence (Sections 4.6 and 6.6). The economics are modeled rather than measured: pilot costs, and the point at which training repays itself, are still to be reported (Sections 5 and 7.4).

9 Conclusion

Within a game, a learner with a referee went beyond the best human play. We asked whether the same recipe can improve the first algorithm a system proposes for a new problem, and made the question testable. Invention becomes a game with a board, a move, a referee and an incumbent opponent. Its price is the cost of a verified win. EurekaZero is designed to train a grammar-constrained writer and a separate problem–program map on targets from verified search, and to deploy one program per problem. The design, the cost model and the evaluation protocol are specified here. The retained development records show admission and search behavior, not transfer.

The decisive experiment is a single comparison: the frozen reference writer against the same writer after a predeclared training intervention, one proposal each, on problem families held out from both (Section 7.2). Engineered intelligence, in our sense, is a capability whose aim, judge and memory are designed to be audited. Whether experience can also make its first guess better is what Section 7 must show.

A Referee Inventory

Section 3 traces the four pillars of Section 2 through the milestones directly needed for our argument: algorithm selection [90], typed genetic programming and evolutionary

search [53, 71, 89], self-play and expert iteration [1, 93, 102], and task-conditioned amortization across tabular problems [34, 44, 73]. Each of those systems depends on a referee. Table 9 collects, domain by domain, the referees that the main text relies on. A domain enters the innovation game only when its referee is reliable and affordable (Section 4.5).

Table 9: **Referees by domain.** Each row restates a referee described in Sections 3 and 4; the last row is the referee of this paper.

Domain	Referee	Systems
Board games	The rules of the game and its final score	AlphaGo Zero, AlphaZero [102, 103]
Machine-learning research	A sealed test set that scores every entry	Netflix Prize, ImageNet [7, 29, 92]
Code	Unit tests; a code executor	Codex, Absolute Zero [19, 123]
Matrix multiplication	The rules of TensorGame	AlphaTensor [33]
Sorting	Sorting tests	AlphaDev [66]
Mathematical constructions	Programmatic evaluators	FunSearch [91]
Accelerator kernels	Timers	AlphaEvolve [75]
Formal proofs	The Lean proof checker	AlphaProof, STP [28, 45]
RNA design	Laboratory tests of designs	EteRNA [60]
Algorithm invention (this paper)	Hidden-fold scores against the incumbent; promotion on strict improvement	EurekaZero (Sections 4.4 and 6)

B Incident Log and Audit Safeguards

Section 6.6 excludes a test-selected search baseline. This appendix records that exclusion and the audit safeguards the evaluation protocol requires, so that artifacts violating held-out isolation are recorded rather than silently dropped:

- **Excluded test-selected search baseline.** An earlier internal leaderboard entry selected each dataset's best test-scored outcome from 63,108 evaluations in total across 42 datasets. Because selecting among candidates using test-fold scores biases the reported metric upward [17], that entry is excluded from all comparative claims.
- **Input-closure and canary checks.** Before benchmark release, the evaluation protocol requires passing receipts for: an inner-fold label-poison test, a sealed canary check, and a train-only input-closure audit verifying that held-out test targets and their derived statistics do not enter board encoding or candidate generation. The final pre-registration and these audit receipts belong to the evaluation record of Section 7.1 and are not reproduced here.

C Training Details

This appendix records the implementation specifications referenced in Section 6 for the historical reference checkpoint (separate from the later adapter snapshot in Section 6.8):

- **Board encoder blocks (Section 6.2).** The frozen 2,048-value context $z_p = \phi_{\mathrm{P}}(p)$ concatenates three blocks: (i) 512 values pooling a 128-dimensional pretrained row encoder across training rows by mean, standard deviation, and 10th and 90th percentiles; (ii) 1,024 values from a projected third-party tabular regressor hidden state capped at 768 context rows and 256 query rows (Foundation, version `foundation_enc.v1.tabpfn`; license: proprietary, available on request; redistribution prohibited); and (iii) 512 values from two frozen target-summary windows initialized with fixed seed 17. Any context with a non-finite entry is refused rather than imputed. Encoding records report wall-clock latency from 11.7 s to 20.8 s across eight reference encoding jobs, within the 30 s cap.
- **Objective terms and parameter split (Sections 6.4 and 6.5).** Lists G_p require at least eight valid scored programs per problem, with percentile $\rho_w = 1 - \mathrm{rank}/(|G_p| - 1)$ and writer sequence weight $0.05 + \rho_w^4$. The native trainer gives problem families equal mass and forms each training comparison from one list: its best program plus one program drawn from each of seven equal rank bands of the rest. Every objective term is computed on these eight-program comparisons, so Equations (3) and (4) show the list-level form. The native sequence term applies label smoothing over legal tokens and averages each program's token loss before taking the normalized weighted mean over programs. The displayed aggregate coefficients match the native objective; the checkpoint's objective identity and configuration still belong in its evaluation record. In the combined objective of Equation (5), the pairwise Bradley–Terry term weights pairs by their score gap, doubled when either program is at or above the 90th percentile. The network has 27,796,494 parameters in total (six decoder blocks, width 512, four attention heads, prefix context token, and maximum length 128 tokens over the 401-token grammar); approximately 1.05 million parameters belong to training-time ranking and control heads that are absent at deployment.
- **Optimization notes (Section 6.5).** The training records describe an AdamW run (learning rate 10^{-4}, weight decay 0.01, gradient-norm clip 1.0, global batch size 2,048) on eight A100 GPUs in September 2026 over 10,214,332 scored programs from 80 training datasets (7,877 lists; 8,894 contexts). These historical reference specifications are separate from the later adapter snapshot in Section 6.8. This paper does not document the evidence on which the checkpoint was accepted; that record and a held-out promotion receipt are required inputs to the evaluation contract (Section 7.1).

- **Execution sandbox and parity (Section 6.6).** The final evaluation protocol requires candidate isolation with seccomp and Landlock, cgroup memory limits with swap disabled, and native versus emitted-code agreement within $10^{-9}\max(1, |\hat{y}|)$. Isolation and parity receipts from the final evaluator belong to the same evaluation record.

References

[1] T. Anthony, Z. Tian, and D. Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, volume 30, pages 5360–5370, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/d8e1344e27a5b08cdfd5d027d9b8d6de-Paper.pdf.

[2] P. Azoulay, C. Fons-Rosen, and J. S. Graff Zivin. Does science advance one funeral at a time? *American Economic Review*, 109(8):2889–2920, 2019. doi: 10.1257/aer.20161574.

[3] L. Bailey, K. Wen, K. Dong, T. Hashimoto, and T. Ma. Scaling self-play with self-guidance. arXiv:2604.20209, April 2026. URL https://arxiv.org/abs/2604.20209.

[4] D. Balduzzi, K. Tuyls, J. Pérolat, and T. Graepel. Re-evaluating evaluation. In *Advances in Neural Information Processing Systems*, volume 31, pages 3268–3279. Curran Associates, 2018. URL https://arxiv.org/abs/1806.02643.

[5] M. Balog, A. L. Gaunt, M. Brockschmidt, S. Nowozin, and D. Tarlow. DeepCoder: Learning to write programs. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1611.01989.

[6] R. Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957.

[7] J. Bennett and S. Lanning. The Netflix Prize. In *Proceedings of KDD Cup and Workshop 2007*, 2007. URL https://www.cs.uic.edu/~liub/KDD-cup-2007/proceedings/The-Netflix-Prize-Bennett.pdf.

[8] N. Bloom, C. I. Jones, J. Van Reenen, and M. Webb. Are ideas getting harder to find? *American Economic Review*, 110 (4):1104–1144, 2020. doi: 10.1257/aer.20180338.

[9] M. A. Boden. *The Creative Mind: Myths and Mechanisms*. Weidenfeld & Nicolson, London, 1990.

[10] M. A. Boden. Creativity and artificial intelligence. *Artificial Intelligence*, 103(1–2):347–356, 1998. doi: 10.1016/S0004-3702(98)00055-1.

[11] J. L. Borges. La biblioteca de Babel. In *El jardín de senderos que se bifurcan*. Sur, Buenos Aires, 1941. Printed 30 December 1941, issued 1942 (LCCN 43002084). English translation by James E. Irby, "The Library of Babel", in *Labyrinths*, New Directions, 1962.

[12] R. A. Bradley and M. E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324.

[13] R. Brandom. What happens when AI starts building itself? TechCrunch, 14 May 2026. URL https://techcrunch.com/2026/05/14/what-happens-when-ai-starts-building-itself/.

[14] T. Brooks, B. Peebles, C. Holmes, W. DePue, Y. Guo, L. Jing, D. Schnurr, J. Taylor, T. Luhman, E. Luhman, C. Ng, R. Wang, and A. Ramesh. Video generation models as world simulators. OpenAI technical report, February 2024. URL https://openai.com/index/video-generation-models-as-world-simulators/.

[15] D. T. Campbell. Blind variation and selective retention in creative thought as in other knowledge processes. *Psychological Review*, 67(6):380–400, 1960. doi: 10.1037/h0040373.

[16] Z. Cao, T. Qin, T.-Y. Liu, M.-F. Tsai, and H. Li. Learning to rank: From pairwise approach to listwise approach. In *Proceedings of the 24th International Conference on Machine Learning (ICML)*, pages 129–136, 2007. doi: 10.1145/1273496.1273513.

[17] G. C. Cawley and N. L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70): 2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

[18] W. G. Chase and H. A. Simon. Perception in chess. *Cognitive Psychology*, 4(1):55–81, 1973. doi: 10.1016/0010-0285(73)90004-2.

[19] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. W. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. A. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. arXiv:2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.

[20] X. Chen, C. Liang, D. Huang, E. Real, K. Wang, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu, and Q. V. Le. Symbolic discovery of optimization algorithms. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2302.06675.

[21] Z. Chen, Y. Deng, H. Yuan, K. Ji, and Q. Gu. Self-play fine-tuning converts weak language models to strong language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, 2024. URL https://arxiv.org/abs/2401.01335.

[22] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1706.03741.

[23] S. Cooper, F. Khatib, A. Treuille, J. Barbero, J. Lee, M. Beenen, A. Leaver-Fay, D. Baker, Z. Popović, and Foldit Players. Predicting protein structures with a multiplayer online game. *Nature*, 466(7307):756–760, 2010. doi: 10.1038/nature09304.

[24] H. Dai, Y. Tian, B. Dai, S. Skiena, and L. Song. Syntax-directed variational autoencoder for structured data. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://arxiv.org/abs/1802.08786.

[25] C. Darwin. *On the Origin of Species by Means of Natural Selection, or the Preservation of Favoured Races in the Struggle for Life*. John Murray, London, 1859. URL http://darwin-online.org.uk/content/frameset?itemID=F373&viewtype=text&pageseq=1.

[26] A. D. de Groot. *Thought and Choice in Chess*. Mouton, The Hague, 1965.

[27] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, et al. DeepSeek-V3 technical report. arXiv:2412.19437, 2024. URL https://arxiv.org/abs/2412.19437.

[28] K. Dong and T. Ma. STP: Self-play LLM theorem provers with iterative conjecturing and proving. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, 2025. URL https://proceedings.mlr.press/v267/dong25h.html.

[29] D. Donoho. 50 years of data science. *Journal of Computational and Graphical Statistics*, 26(4):745–766, 2017. doi: 10.1080/10618600.2017.1384734.

[30] S. Du, X. Yan, J. Shi, Z. Cao, S. Feng, Z. Liang, B. Sun, T. Peng, Y. Zhou, X. Li, J. Zhou, L. He, B. Zhang, and L. Bai. MLEvolve: A self-evolving framework for automated machine learning algorithm discovery. arXiv:2606.06473, June 2026. URL https://arxiv.org/abs/2606.06473.

[31] K. Ellis, C. Wong, M. Nye, M. Sablé-Meyer, L. Morales, L. Hewitt, L. Cary, A. Solar-Lezama, and J. B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 835–850, 2021. doi: 10.1145/3453483.3454080.

[32] A. E. Elo. *The Rating of Chessplayers, Past and Present*. Arco, New York, 1978.

[33] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz, D. Silver, D. Hassabis, and P. Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022. doi: 10.1038/s41586-022-05172-4.

[34] M. Fey, V. Kocijan, F. Lopez, J. E. Lenssen, and J. Leskovec. KumoRFM: A relational foundation model. Kumo Technical Report, May 2025. URL https://docs.nvidia.com/sdgm/research/concepts-models/kumorfm-paper.

[35] R. M. Friedberg. A learning machine: Part I. *IBM Journal of Research and Development*, 2(1):2–13, 1958. doi: 10.1147/rd.21.0002.

[36] I. J. Good. Speculations concerning the first ultraintelligent machine. In F. L. Alt and M. Rubinoff, editors, *Advances in Computers*, volume 6, pages 31–88. Academic Press, 1966. doi: 10.1016/S0065-2458(08)60418-0.

[37] J. Gottweis, W.-H. Weng, A. Daryin, T. Tu, P. Sirkovic, A. Myaskovsky, G. Glowaty, F. Weissenberger, A. Orlandi, D. Popovici, A. Palepu, K. Rong, R. Tanno, K. Saab, F. Zhang, J. Blum, A. Carroll, K. Kulkarni, N. Tomasev, D. Zverinski, I. Rendulic, E. Vedadi, F. Hasler, L. Rimanic, M. Boia, I. Budiselic, B. Feinstein, M. Bellaiche, T. Sheffer, J. Freyberg, J. Ratcliff, O. Bertolli, K. Chou, A. Hassidim, B. Gokturk, A. Vahdat, Y. Guan, V. Dhillon, E. D. Vaishnav, B. Lee, T. R. D. Costa, J. R. Penadés, G. Peltz, Y. Matias, J. Manyika, D. Hassabis, Y. Xu, P. Kohli, A. Pawlosky, A. Karthikesalingam, and V. Natarajan. Accelerating scientific discovery with Co-Scientist. *Nature*, 655(8122):487–496, 2026. doi: 10.1038/s41586-026-10644-y. Preprint: arXiv:2502.18864 (2025), "Towards an AI co-scientist".

[38] S. Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 317–330, 2011. doi: 10.1145/1926385.1926423.

[39] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.

[40] R. W. Hamming. You and your research. Bell Communications Research Colloquium Seminar, Morris Research and Engineering Center, Morristown, NJ, 1986. URL https://hdl.handle.net/10945/37504. Talk given 7 March 1986; transcribed by J. F. Kaiser.

[41] D. Hassabis and A. Kantrowitz. Google DeepMind CEO Demis Hassabis: The path to AGI, LLM creativity, and Google smart glasses. Big Technology Podcast, 23 January 2025. URL https://www.bigtechnology.com/p/google-deepmind-ceo-demis-hassabis.

[42] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030, 2022. URL https://arxiv.org/abs/2203.15556.

[43] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975.

[44] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025. doi: 10.1038/s41586-024-08328-6.

[45] T. Hubert, R. Mehta, L. Sartran, M. Z. Horváth, G. Žužić, E. Wieser, A. Huang, J. Schrittwieser, Y. Schroecker, H. Masoom, O. Bertolli, T. Zahavy, A. Mandhane, J. Yung, I. Beloshapka, B. Ibarz, V. Veeriah, L. Yu, O. Nash, P. Lezeau, S. Mercuri, C. Sönne, B. Mehta, A. Davies, D. Zheng, F. Pedregosa, Y. Li, I. von Glehn, M. Rowland, S. Albanie, A. Velingker, S. Schmitt, E. Lockhart, E. Hughes, H. Michalewski, N. Sonnerat, D. Hassabis, P. Kohli, and D. Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026. doi: 10.1038/s41586-025-09833-y. Published online 12 November 2025.

[46] B. F. Jones. The burden of knowledge and the "death of the Renaissance man": Is innovation getting harder? *Review of Economic Studies*, 76(1):283–317, 2009. doi: 10.1111/j.1467-937X.2008.00531.x.

[47] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek,

A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596 (7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

[48] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. arXiv:2001.08361, 2020. URL https://arxiv.org/abs/2001.08361.

[49] A. Karpathy. `autoresearch`: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 6 March 2026. URL https://github.com/karpathy/autoresearch.

[50] F. Khatib, F. DiMaio, Foldit Contenders Group, Foldit Void Crushers Group, S. Cooper, M. Kazmierczyk, M. Gilski, S. Krzywda, H. Zábranská, I. Pichová, J. Thompson, Z. Popović, M. Jaskolski, and D. Baker. Crystal structure of a monomeric retroviral protease solved by protein folding game players. *Nature Structural & Molecular Biology*, 18(10): 1175–1177, 2011. doi: 10.1038/nsmb.2119.

[51] Kimi Team. Kimi K2: Open agentic intelligence. arXiv:2507.20534, 2025. URL https://arxiv.org/abs/2507.20534.

[52] A. Koestler. *The Act of Creation*. Hutchinson, London, 1964.

[53] J. R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, 1992. URL https://mitpress.mit.edu/9780262111706/genetic-programming/.

[54] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, pages 1097–1105, 2012. URL https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.

[55] T. S. Kuhn. *The Structure of Scientific Revolutions*. University of Chicago Press, Chicago, 1962.

[56] M. J. Kusner, B. Paige, and J. M. Hernández-Lobato. Grammar variational autoencoder. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1945–1954, 2017. URL https://proceedings.mlr.press/v70/kusner17a.html.

[57] T. Lattimore and C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, Cambridge, 2020. doi: 10.1017/9781108571401.

[58] Y. LeCun and A. Kantrowitz. Why can't AI make its own discoveries? — with Yann LeCun. Big Technology Podcast, 19 March 2025. URL https://podscripts.co/podcasts/big-technology-podcast/why-cant-ai-make-its-own-discoveries-with-yann-lecun.

[59] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.

[60] J. Lee, W. Kladwang, M. Lee, D. Cantu, M. Azizyan, H. Kim, A. Limpaecher, S. Yoon, A. Treuille, R. Das, and EteRNA Participants. RNA design rules from a massive open laboratory. *Proceedings of the National Academy of Sciences*, 111(6):2122–2127, 2014. doi: 10.1073/pnas.1313039111.

[61] Y.-J. Lee, S. Kim, M. Kang, A. C. L. Chuen, Z. Chen, S. Han, T. Jung, and D. Kang. Evolution Fine-Tuning: Learning to discover across 371 optimization tasks. arXiv:2606.29082, June 2026. URL https://arxiv.org/abs/2606.29082.

[62] J. Lehman and K. O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011. doi: 10.1162/EVCO_a_00025.

[63] D. B. Lenat. *AM: An Artificial Intelligence Approach to Discovery in Mathematics as Heuristic Search*. PhD thesis, Stanford University, 1976. Stanford AI Lab Memo AIM-286; report STAN-CS-76-570.

[64] D. B. Lenat. EURISKO: A program that learns new heuristics and domain concepts. The nature of heuristics III: Program design and results. *Artificial Intelligence*, 21(1–2):61–98, 1983. doi: 10.1016/S0004-3702(83)80005-8.

[65] C. Lu, C. Lu, R. T. Lange, Y. Yamada, S. Hu, J. Foerster, D. Ha, and J. Clune. Towards end-to-end automation of AI research. *Nature*, 651(8107):914–919, 2026. doi: 10.1038/s41586-026-10265-5. Preprint: arXiv:2408.06292 (2024), "The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery".

[66] D. J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent, S. Iqbal, J.-B. Lespiau, A. Ahern, T. Köppe, K. Millikin, S. Gaffney, S. Elster, J. Broshear, C. Gamble, K. Milan, R. Tung, M. Hwang, T. Cemgil, M. Barekatain, Y. Li, A. Mandhane, T. Hubert, J. Schrittwieser, D. Hassabis, P. Kohli, M. Riedmiller, O. Vinyals, and D. Silver. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618 (7964):257–263, 2023. doi: 10.1038/s41586-023-06004-9.

[67] H. Massalin. Superoptimizer: A look at the smallest program. In *Proceedings of the 2nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS II)*, pages 122–126, 1987. doi: 10.1145/36206.36194.

[68] A. Merchant, S. Batzner, S. S. Schoenholz, M. Aykol, G. Cheon, and E. D. Cubuk. Scaling deep learning for materials discovery. *Nature*, 624(7990):80–85, 2023. doi: 10.1038/s41586-023-06735-9.

[69] C. Metz. In two moves, AlphaGo and Lee Sedol redefined the future. *Wired*, 16 March 2016. URL https://www.wired.com/2016/03/two-moves-alphago-lee-sedol-redefined-future/.

[70] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.

[71] D. J. Montana. Strongly typed genetic programming. *Evolutionary Computation*, 3(2):199–230, 1995. doi: 10.1162/evco.1995.3.2.199.

[72] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites. arXiv:1504.04909, 2015. URL https://arxiv.org/abs/1504.04909.

[73] A. Müller, C. Curino, and R. Ramakrishnan. MotherNet: Fast training and inference via hyper-network transformers. In

International Conference on Learning Representations (ICLR), 2025. URL https://arxiv.org/abs/2312.08598.

[74] J. Nielsen and J. D. Keasling. Engineering cellular metabolism. *Cell*, 164(6):1185–1197, 2016. doi: 10.1016/j.cell.2016.02.004.

[75] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131, 2025. URL https://arxiv.org/abs/2506.13131.

[76] NVIDIA. NVIDIA H100 tensor core GPU: Specifications. NVIDIA Data Center Documentation, 2026. URL https://www.nvidia.com/en-us/data-center/h100/.

[77] J. Oh, G. Farquhar, I. Kemaev, D. A. Calian, M. Hessel, L. Zintgraf, S. Singh, H. van Hasselt, and D. Silver. Discovering state-of-the-art reinforcement learning algorithms. *Nature*, 648(8093):312–319, 2025. doi: 10.1038/s41586-025-09761-x.

[78] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022. URL https://arxiv.org/abs/2203.02155.

[79] M. Park, E. Leahey, and R. J. Funk. Papers and patents are becoming less disruptive over time. *Nature*, 613(7942): 138–144, 2023. doi: 10.1038/s41586-022-05543-x.

[80] D. Patel and D. Amodei. Dario Amodei (Anthropic CEO) — the hidden pattern behind every AI breakthrough. Dwarkesh Podcast, 8 August 2023. URL https://www.dwarkesh.com/p/dario-amodei.

[81] C. S. Peirce. *Collected Papers of Charles Sanders Peirce, Volume V: Pragmatism and Pragmaticism*. Harvard University Press, Cambridge, MA, 1934. Edited by Charles Hartshorne and Paul Weiss.

[82] B. Pfahringer, H. Bensusan, and C. Giraud-Carrier. Meta-learning by landmarking various learning algorithms. In *Proceedings of the 17th International Conference on Machine Learning (ICML)*, pages 743–750. Morgan Kaufmann, 2000. URL https://ml.cms.waikato.ac.nz/publications/2000/landmarking_icml2000.pdf.

[83] H. Poincaré. Mathematical creation. *The Monist*, 20(3): 321–335, 1910. doi: 10.1093/monist/20.3.321. Translated by George Bruce Halsted.

[84] M. Polanyi. *The Tacit Dimension*. Doubleday, Garden City, NY, 1966.

[85] G. Pólya. *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, Princeton, NJ, 1945.

[86] K. R. Popper. *The Logic of Scientific Discovery*. Hutchinson, London, 1959.

[87] K. R. Popper. *Conjectures and Refutations: The Growth of Scientific Knowledge*. Routledge & Kegan Paul, London, 1963.

[88] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763, 2021. URL https://proceedings.mlr.press/v139/radford21a.html.

[89] E. Real, C. Liang, D. R. So, and Q. V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In H. Daumé III and A. Singh, editors, *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 8007–8019. PMLR, 2020. URL https://proceedings.mlr.press/v119/real20a.html.

[90] J. R. Rice. The algorithm selection problem. In M. Rubinoff and M. C. Yovits, editors, *Advances in Computers*, volume 15, pages 65–118. Academic Press, New York, 1976. doi: 10.1016/S0065-2458(08)60520-3.

[91] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

[92] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3): 211–252, 2015. doi: 10.1007/s11263-015-0816-y.

[93] A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959. doi: 10.1147/rd.33.0210.

[94] J. Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. arXiv:cs/0309048, 2003. URL https://arxiv.org/abs/cs/0309048.

[95] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. doi: 10.1038/s41586-020-03051-4.

[96] C. E. Shannon. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950. doi: 10.1080/14786445008521796.

[97] M. Shin, J. Kim, B. van Opheusden, and T. L. Griffiths. Superhuman artificial intelligence can improve human decision-making by increasing novelty. *Proceedings of the National Academy of Sciences*, 120(12):e2214840120, 2023. doi: 10.1073/pnas.2214840120.

[98] C. Si, D. Yang, and T. Hashimoto. Can LLMs generate novel research ideas? A large-scale human study with 100+ NLP researchers. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=M23dTGWCZy.

[99] C. Si, T. Hashimoto, and D. Yang. The ideation–execution gap: Execution outcomes of LLM-generated versus human research ideas. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=Fllp8l6Puy.

[100] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.

[101] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv:1712.01815, 2017. URL https://arxiv.org/abs/1712.01815.

[102] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017. doi: 10.1038/nature24270.

[103] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404.

[104] D. K. Simonton. Creativity and discovery as blind variation: Campbell's (1960) BVSR model after the half-century mark. *Review of General Psychology*, 15(2):158–174, 2011. doi: 10.1037/a0022912.

[105] R. Socher. AI's biggest breakthrough won't be a chatbot. Next Big Idea Club, Book Bites, 8 October 2026. URL https://nextbigideaclub.com/magazine/ais-biggest-breakthrough-wont-chatbot-bookbite/61534/.

[106] R. Socher. *The Eureka Machine: Why AI Is the Key to Unlocking a New Era of Scientific Discoveries*. PublicAffairs, New York, 2026. ISBN 9781541705708. URL https://www.hachettebookgroup.com/titles/richard-socher/the-eureka-machine/9781541705708/.

[107] J. M. Stokes, K. Yang, K. Swanson, W. Jin, A. Cubillos-Ruiz, N. M. Donghia, C. R. MacNair, S. French, L. A. Carfrae, Z. Bloom-Ackermann, V. M. Tran, A. Chiappino-Pepe, A. H. Badran, I. W. Andrews, E. J. Chory, G. M. Church, E. D. Brown, T. S. Jaakkola, R. Barzilay, and J. J. Collins. A deep learning approach to antibiotic discovery. *Cell*, 180(4): 688–702.e13, 2020. doi: 10.1016/j.cell.2020.01.021.

[108] A. Surina, A. Mansouri, L. Quaedvlieg, A. Seddas, M. Viazovska, E. Abbe, and C. Gulcehre. Algorithm discovery with LLMs: Evolutionary search meets reinforcement learning. arXiv:2504.05108, 2025. URL https://arxiv.org/abs/2504.05108.

[109] R. S. Sutton. The bitter lesson. Essay, 13 March 2019. URL http://www.incompleteideas.net/IncIdeas/BitterLesson.html.

[110] G. Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995. doi: 10.1145/203330.203343.

[111] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.

[112] A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. arXiv:1807.03748, 2018. URL https://arxiv.org/abs/1807.03748.

[113] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017. URL https://arxiv.org/abs/1706.03762.

[114] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, and D. Silver. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019. doi: 10.1038/s41586-019-1724-z.

[115] S. Von Arx, L. Chan, and B. Barnes. Recent frontier models are reward hacking. METR blog, 5 June 2025. URL https://metr.org/blog/2025-06-05-recent-reward-hacking/.

[116] G. Wallas. *The Art of Thought*. Jonathan Cape, London, 1926.

[117] K. Wiggers. Sakana walks back claims that its AI can dramatically speed up model training. TechCrunch, 21 February 2025. URL https://techcrunch.com/2025/02/21/sakana-walks-back-claims-that-its-ai-can-dramatically-speed-up-model-training/.

[118] T. Wolf. The Einstein AI model. Technical blog essay, February 2025. URL https://thomwolf.io/blog/scientific-ai.html.

[119] D. H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996. doi: 10.1162/neco.1996.8.7.1341.

[120] M. Yuksekgonul, D. Koceja, X. Li, F. Bianchi, J. McCaleb, X. Wang, J. Kautz, Y. Choi, J. Zou, C. Guestrin, and Y. Sun. Learning to discover at test time. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. URL https://arxiv.org/abs/2601.16175.

[121] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai. Self-Taught Optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2310.02304.

[122] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. arXiv:2505.22954, 2025. URL https://arxiv.org/abs/2505.22954.

[123] A. Zhao, Y. Wu, Y. Yue, T. Wu, Q. Xu, M. Lin, S. Wang, Q. Wu, Z. Zheng, and G. Huang. Absolute Zero: Reinforced self-play reasoning with zero data. arXiv:2505.03335, 2025. URL https://arxiv.org/abs/2505.03335.

[124] Z. Zhong, I. Segal, I. Bercovich, S. Saxena, K. Zhang, and A. Raghunathan. Hardening agent benchmarks with adversarial hacker-fixer loops. arXiv:2606.08960, 2026. URL https://arxiv.org/abs/2606.08960.

[125] B. Zoph and Q. V. Le. Neural architecture search with reinforcement learning. In *International Conference on*

Learning Representations (ICLR), 2017. URL https://arxiv.org/abs/1611.01578.